

ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ
ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ

ΣΥΣΤΗΜΑΤΑ ΠΑΡΑΛΛΗΛΗΣ ΕΠΕΞΕΡΓΑΣΙΑΣ

Εξαμηνιαία Άσκηση

Ομάδα: Parlab 24

Φίλιππος Σεβαστάκης 03119183
Κωνσταντίνος Ιωάννου 03119840
Γεώργιος Μυτιληναίος 03119841

Χειμερινό εξάμηνο 2023/24

Ενότητα 1: Εξοικείωση με το περιβάλλον προγραμματισμού

Αποτελέσματα, συγκρίσεις και σχόλια

Αρχικά παραλληλοποιούμε κατάλληλα τον κώδικα που μας δόθηκε και υλοποιεί σειριακά την εφαρμογή "Game of Life", ώστε η "δουλειά" των εμφωλιασμένων επανα- λήψεων να διαχωρίζεται σε πολλαπλούς πυρήνες επιταχύνοντας έτσι την διαδικασία. Συγκεκριμένα έχουμε ενα perfect-loop στο οποίο έχει νόημα να παραλληλοποιήσουμε το δεύτερο ή/και το τρίτο loop καθώς αυτά είναι που σκανάρουν το επίπεδο του game of life κάθε γραμμή και κάθε στήλη αντίστοιχα. Αντιθέτως, το εξωτερικό loop αποτυπώνει καθε time-frame και επομένως δεν γίνεται να παραλληλοποιήθει, καθώς η κατάσταση των σημείων σε μία χρονική στιγμή είναι συνάρτηση της κατάστασης των γειτόνων αυτής την προηγούμενη χρονική στιγμή. Εμείς επιλέξαμε συγκεκριμένα να παραλληλοποιήσουμε το δεύτερο εμφωλιασμένο loop, χρησιμοποιώντας το εργαλείο Open MP, αφού πρόκειται για μοντέλο κοινού χώρου διευθύνσεων, όπως φαίνεται στον κώδικα που παρατίθεται παρακάτω. Αξίζει να σημειωθεί ότι τις μεταβλητές nbrs, i και j χρειάζεται να τις ορίσουμε ως private (κάθε νήμα έχει για την ίδια μεταβλητή ένα "ιδιωτικό" αντίγραφο), καθώς σε άλλη περίπτωση - δεδομένου του shared address space - συμβαίνουν race conditions (δηλαδή ένα νήμα αλλάζει την "κοινή" μεταβλητή όσο ένα άλλο νήμα την διαβάζει).

```
int main (int argc, char *argv[])
{
        int nthreads, tid;

        /* Fork a team of threads giving them their own copies of variables */
        #pragma omp parallel private(nthreads, tid)
        {
                /* Obtain thread number */
                tid = omp_get_thread_num();
                printf("Hello World from thread = %d\n", tid);

                /* Only master thread does this */
                if (tid == 0) {
                        nthreads = omp_get_num_threads();
                        printf("Number of threads = %d\n", nthreads);
                }

        }  /* All threads join master thread and disband */
        return 0;
}
```

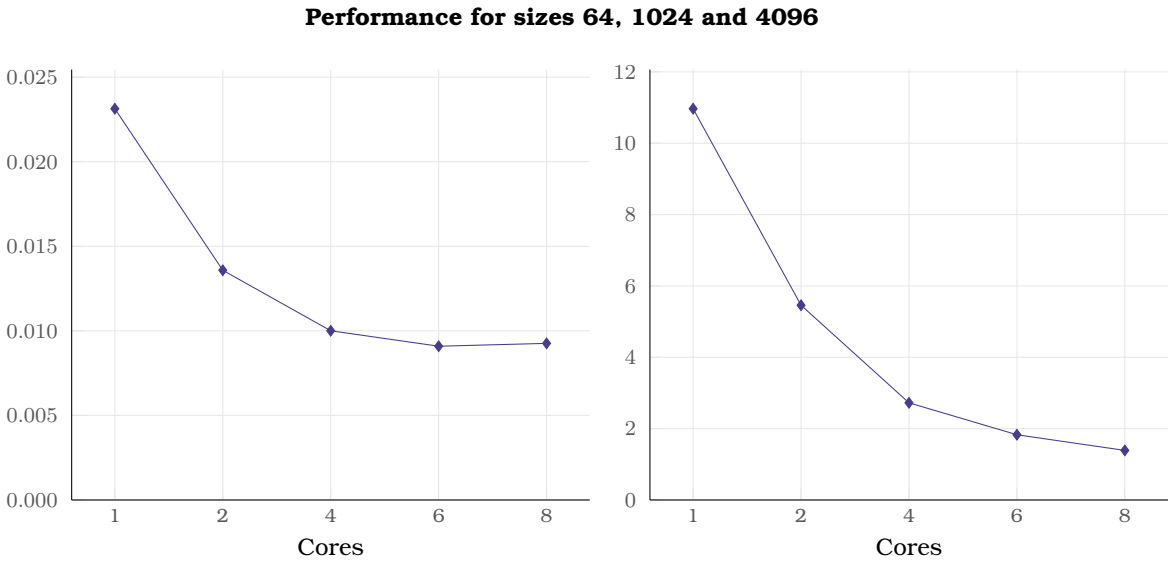
Listing 1: Code snippet of the critical section

Εκτελέσαμε, λοιπόν, τον παραπάνω κώδικα για διάφορο πλήθος πυρήνων και για διαφορετικού μεγέθους εφαρμογές, και παρακάτω έχουμε συγκεντρώσει τους αντίστοιχους χρόνους εκτέλεσης

Cores	**Problem size (N)**		
	64	1024	4096
1	0.023134	10.967618	175.908166
2	0.013582	5.458978	88.792250
4	0.010005	2.723239	44.540773
6	0.009092	1.828639	37.103234
8	0.009260	1.389687	36.495657

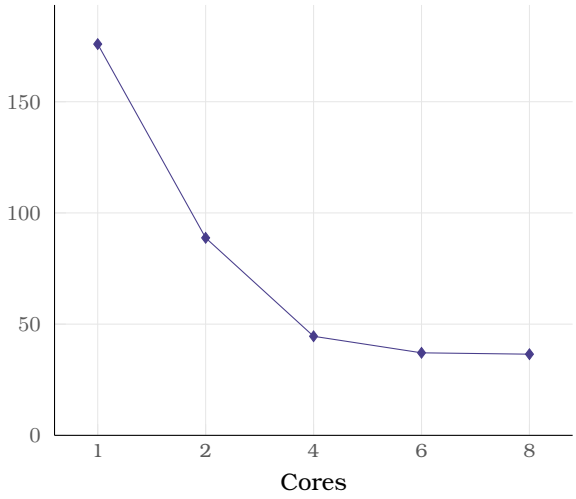
Figure 1: Game Of Life - Execution Time for different number of cores and problem sizes

Τα αντίστοιχα διαγράμματα των *performance*, *speedup* και *efficiency* από τα παραπάνω δεδομένα είναι τα εξής:



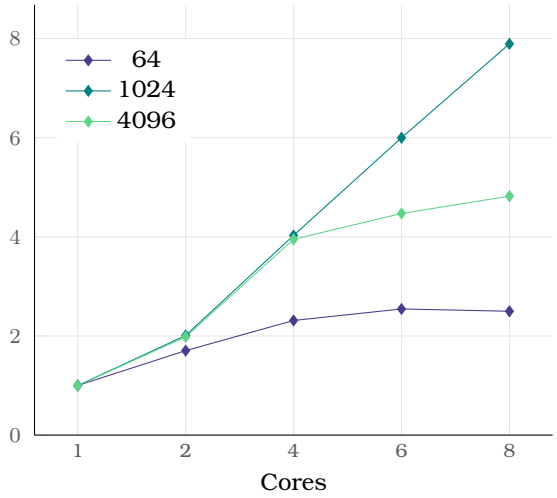
(a) Performance for size 64

(b) Performance for size 1024

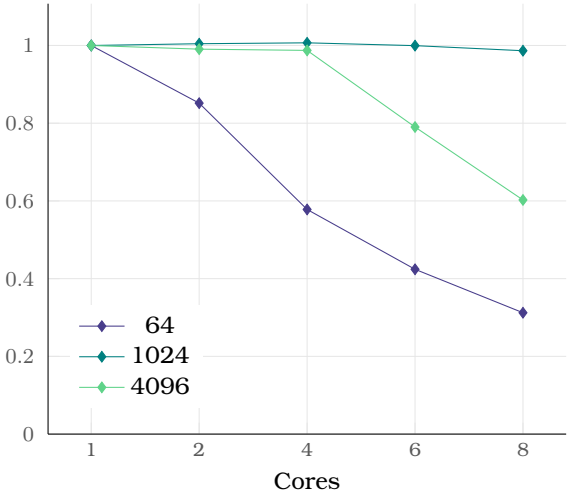


(c) Performance for size 4096

Speedup and Efficiency graphs



(d) Speedup for problem sizes 64, 1024 and 4096



(e) Efficiencyfor problem sizes 64, 1024 and 4096

Εκ πρώτης όψεως, φαίνεται ότι για όλες τις υλοποιήσεις (δηλαδή για τα μεγέθη του προβλήματος 64,1024,4096) έχουμε βελτίωση του χρόνου εκτέλεσης με την αύξηση του αριθμού των πυρήνων, ωστόσο όλο και μειούμενη με εκθετικό ρυθμό. Από τα διαγράμματα, κυρίως του speedup (S = T1/Tn, n = number of cores) και του efficiency, μπορούμε να παρατηρήσουμε όμως πιο συγκεκριμένα τα εξής:

Για N=64: Για παραπάνω από 4 πυρήνες η βελτίωση δεν είναι σημαντική και μάλιστα για 8 πυρήνες βλέπουμε ότι ο χρόνος εκτέλεσης έχει αυξηθεί λίγο σε σχέση με αυτόν που είχαμε για 6. Αυτό μπορεί να δικαιολογηθεί, μεταξύ άλλων, από το γεγονός ότι το πρόηβλημα και η παραλληλοποίηση που κάνουμε σε αυτό είναι αμελητέα μπροστά στον χρόνο δημιουργίας των threads και του συγχρονισμού αυτών, οπότε και η προσθήκη επιπλέον πυρήνων δεν βελτιώνει τον χρόνο εκτέλεσης και το speedup, πόσο μάλλον μπορεί και να τα χειροτερέψει (πχ μεταξύ των 6 και των 8 πυρήνων).

Για N=4096: Αντίστοιχα, το speedup και αυτού του προβλήματος αυξάνεται μέχρι τους 6 πυρήνες, από τους οποίους και μετά, όμως, δείχνει να συγκλίνει. Αυτό οφείλεται, ενδεχομένως, στις πολλαπλές και χρονοβόρες προσβάσεις στην κύρια μνήμη, που γίνονται λόγω της περιορισμένης μπροστά στο μέγεθος του προβλήματος cache, οι οποίες με τη σειρά τους αυξάνουν την πιθανότητα (ειδικά για όσο περισσότερα cores έχουμε) για race condition στην κύρια μνήμη.

Για N=1024: Εδώ παρατηρούμε, αντιθέτως, γραμμική αύξηση του speedup το οποίο σημαίνει ότι αξίζει να παραλληλοποιήσουμε και με τους οκτώ πυρήνες, ενδεχομένως και περισσότερους (μέχρι προφανώς κάποιο άνω όριο), το οποίο συμβαίνει διότι το μέγεθος του προβλήματος αυτού είναι αρκετά μεγάλο ώστε τα οφέλη της παραλληλοποίησης των υπολογισμών να είναι σημαντικά, μπροστά στη δημιουργία και συγχρονισμό των threads, και ταυτόχρονα αρκετά μικρό ώστε οι caches να μην οδηγούν σε πολλά misses και συνεπώς πολλές προσβάσεις της κύριας μνήμης.

Ενότητα 2: Παραλληλοποίηση και βελτιστοποίηση αλγορίθμων σε αρχιτεκτονικές κοινής μνήμης

Παραλληλοποίηση και βελτιστοποίηση του αλγορίθμου K-means

Υλοποίηση με προσθήκη εντολών συγχρονισμού

Στο σημείο αυτό παραλληλοποιούμε τον αλγόριθμο k-means σε αρχιτεκτονική κοινής μνήμης, χρησιμοποιώντας για τον συγχρονισμό των νημάτων στα critical sections την εντολή #pragma omp atomic. Αντίστοιχα, θα μπορούσαμε για τον ίδιο σκοπό να χρησιμοποιήσουμε την εντολή #pragma omp critical, ωστόσο γνωρίζουμε ότι οι ατομικές εντολές είναι αποδοτικότερες (συγκριτικά με τα locks που υλοποιεί η δεύτερη), όταν αυτές μπορούν να χρησιμοποιηθούν (memory writes).

Ως critical sections, όπως φαίνεται παρακάτω, έχουμε αναγνωρίσει, αρχικά, την προσαύξηση κατά 1 της μεταβλητής delta, καθώς είναι κοινή για όλα τα threads, και του array newClusterSize[index], καθώς διαφορετικά threads μπορούν να έχουν την ίδια τιμή index (find_nearest_cluster) και να συμβούν race conditions. Επιπλέον, critical section έχουμε και στην ανανέωση των τιμών της newClusters, καθώς αντίστοιχα με πριν διαφορετικά threads μπορούν να πάνε να γράψουν ταυτόχρονα σε ίδιες θέσεις του πίνακα (index+numCoords + j).

```
    #pragma omp parallel for private(i,j,index)
    for (i=0; i<numObjs; i++) {
        // find the array index of nearest cluster center
        index = find_nearest_cluster(numClusters, numCoords, &objects[i*numCoords],
    clusters);

        // if membership changes, increase delta by 1
        if (membership[i] != index)
            #pragma omp atomic
            delta += 1.0;

        // assign the membership to object i
        membership[i] = index;

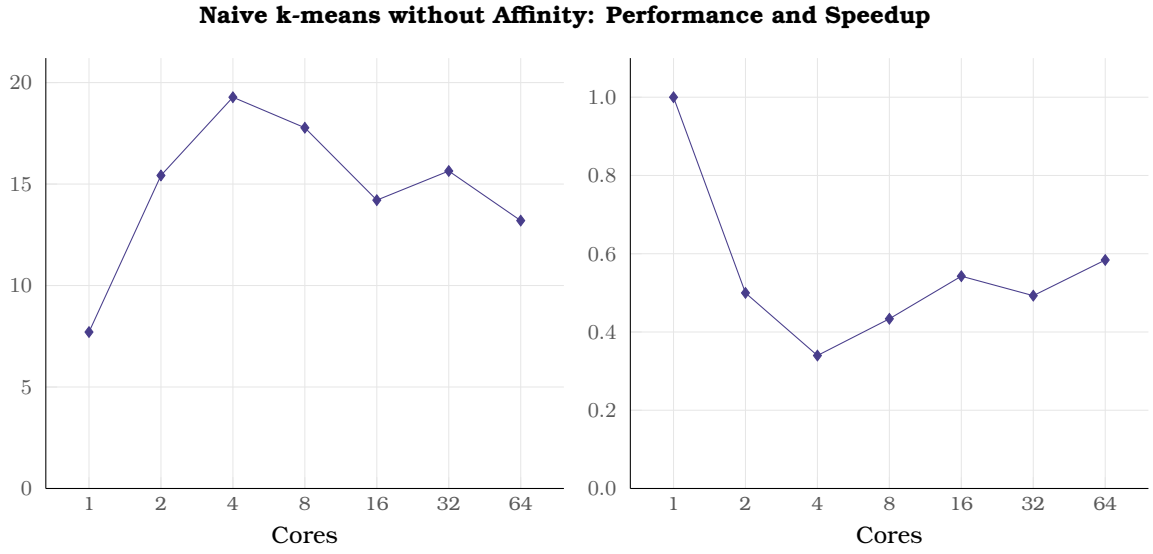
        // update new cluster centers : sum of objects located within
        /*
         * TODO: protect update on shared "newClusterSize" array
         */
        #pragma omp atomic
        newClusterSize[index]++;
        for (j=0; j<numCoords; j++)
            /*
             * TODO: protect update on shared "newClusters" array
             */
            #pragma omp atomic
            newClusters[index*numCoords + j] += objects[i*numCoords + j];
    }
```

Listing 2: Naive-k-means code snippet

Τα αποτελέσματα της παραπάνω υλοποίησης είναι τα εξής:

Cores	Execution Time
1	7.7083
2	7.6285
4	4.8047
8	4.2705
16	8.5747
32	13.4740
64	11.9208

Figure 2: Kmeans naive without Affinity - Execution Time for different number of cores



(a) Performance (Execution time in sec)

(b) Speedup

Με παράλληλη υλοποίηση θα αναμέναμε καλύτερους χρόνους εκτέλεσης, ωστόσο παρατηρούμε από τα αποτελέσματα (διαγράμματα Performance και Speedup) ότι αυτό όχι μόνο δεν συμβαίνει, αλλά έχουμε σημαντική αύξηση αυτών, ήδη με μόλις 2 cores.

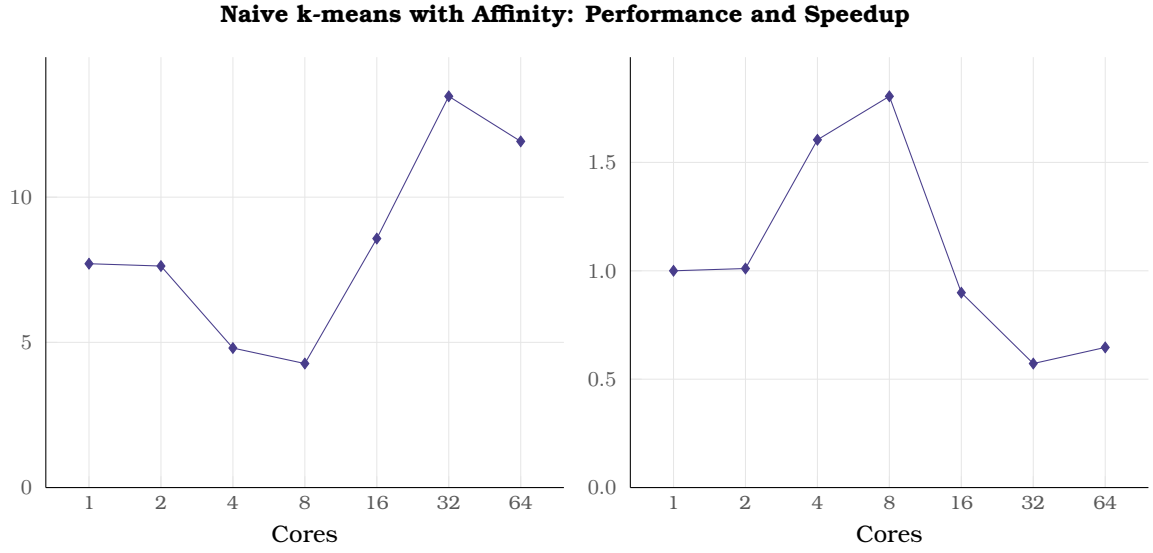
Ένας παράγοντας για το παραπάνω είναι οτι τα critical sections αποτελούν μεγάλο μέρος του προς-παραλληλοποίηση-κώδικα, το οποίο η χρήση των ατομικών εντολών σειριοποιεί. Έτσι, στην πραγματικότητα το μέρος που παραλληλοποιείται είναι πολύ μικρό σε σχέση με το σειριακό (στο οποίο περιλαμβάνονται οι εγγραφές και αναγνώσεις στην και από την μνήμη).
Άλλος ένας λόγος είναι οι επιπλέον καθυστερήσεις που περιλαμβάνει η υλοποίηση, όπως πρωτίστως το κόστος των ατομικών εντολών (αντίστοιχα των locks αν υλοποιούσαμε #pragma omp critical), καθώς και το κόστος της δημιουργίας των threads.

Ένας τελευταίος σημαντικός παράγοντας είναι η έλλειψη κάποιας thread-to-cpu-bind "πολιτικής":

By default, κάθε νήμα μπορεί να δρομολογηθεί σε οποιονδήποτε διαθέσιμο πυρήνα. Έτσι, όμως, οδηγείται σε cache misses, τα οποία θα αποφεύγονταν εάν το κάθε thread ήταν "προσδεμένο" σε συγκεκριμένο πυρήνα και κατ'επέκταση σε συγκεκριμένη cache, για τα δεδομένα της οποίας θα ήταν μόνο αυτό υπεύθυνο. Την λύση στο παραπάνω μας την δίνει η μεταβλητή περιβάλλοντος 'GOMP_CPU_AFFINITY' του OpenMP, με την οποία μπορούμε να εξασφαλίσουμε αυτή την ανάθεση των νημάτων σε συγκεκριμένο πυρήνα. Παρακάτω παρατίθενται τα αποτελέσματα της ίδιας υλοποίησης, συμπεριλαμβάνοντας όμως την μεταβλητή περιβάλλοντος 'GOMP_CPU_AFFINITY':

Cores	**Execution Time**
1	7.7083
2	7.6285
4	4.8047
8	4.2705
16	8.5747
32	13.4740
64	11.9208

Figure 3: Kmeans naive with Affinity - Execution Time for different number of cores



(a) Performance (Execution time in sec)

(b) Speedup

Παρατηρούμε ότι όντως με την προσθήκη της μεταβλητής περιβάλλοντος για το affinity οι χρόνοι με την παραλληλοποίηση είναι καλύτεροι, συγκριτικά με αυτούς χωρίς την μεταβλητή, και μάλιστα μέχρι και τα 8 νήματα έχουμε βελτίωση σε σχέση με το σειριακό (speedup > 1). Ωστόσο, μετά τα 8 νήματα όχι μόνο δεν έχουμε βελτίωση, αλλά μάλιστα οι χρόνοι εκτέλεσης αυξάνονται και γίνονται χειρότεροι,

ακόμα και από αυτόν του σειριακού. Αυτό οφείλεται στο γεγονός οτι από αυτόν τον αριθμό των νημάτων και έπειτα το κέρδος της παραλληλοποίησης του (μικρού συγκριτικά με το σειριακό μέρος) κώδικα είναι μικρότερο από το κόστος που προκύπτει από τον ανταγωνισμό των νημάτων για στην κοινή δομή.

Υλοποίηση με copied πίνακες and reduce

Στο σημείο αυτό θα παραλληλοποιήσουμε τον ίδιο αλγόριθμο, αλλά εκμεταλλευόμενοι την λογική του map-reduce, αντί της χρήσης atomic-operations/locks. Συγκεκριμένα, δημιουργούμε private αντίγραφα των shared δεδομένων για κάθε νήμα, οπότε και αποφεύγουμε την εμφάνιση race conditions, και στο τέλος εκτελούμε κατάλληλο reduction στις κοινές μεταβλητές.

```
    delta = 0.0;
    double local_delta[nthreads];
    /*
     * TODO: Initiliaze local cluster data to zero (separate for each thread)
     */
    for (k=0; k<nthreads; k++) {
        for (i=0; i<numClusters; i++) {
            for (j=0; j<numCoords; j++)
                local_newClusters[k][i*numCoords + j] = 0.0;
            local_newClusterSize[k][i] = 0;
        }
    }

    for (k=0; k<nthreads; k++) local_delta[k]=0.0;

    int tid;

    #pragma omp parallel for private(i,j,index,tid)

    for (i=0; i<numObjs; i++) {
        // find the array index of nearest cluster center
        index = find_nearest_cluster(numClusters, numCoords, &objects[i*numCoords],
    clusters);

        tid = omp_get_thread_num();
        // if membership changes, increase delta by 1
        if (membership[i] != index)
            local_delta[tid] += 1.0;

        // assign the membership to object i
        membership[i] = index;

        // update new cluster centers : sum of all objects located within (average will
    be performed later)
        /*
         * TODO: Collect cluster data in local arrays (local to each thread)
         *       Replace global arrays with local per-thread
         */
        local_newClusterSize[tid][index]++;
        for (j=0; j<numCoords; j++)
            local_newClusters[tid][index*numCoords + j] += objects[i*numCoords + j];

    }

    /*
     * TODO: Reduction of cluster data from local arrays to shared.
```

```
     *       This operation will be performed by one thread
     */

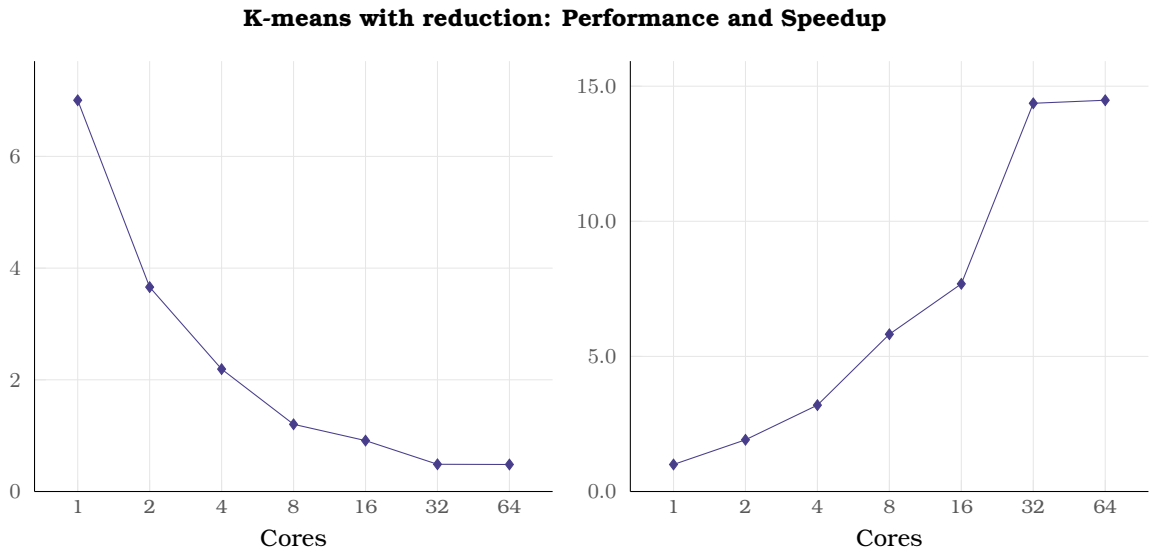
    // reduction
    for (k=0; k<nthreads; k++) {
        for (i=0; i<numClusters; i++) {
            for (j=0; j<numCoords; j++)
                newClusters[i*numCoords + j] += local_newClusters[k][i*numCoords + j];
            newClusterSize[i] += local_newClusterSize[k][i];
        }
        delta += local_delta[k];
    }
```

Listing 3: Reduction-k-means code snippet

Τα αποτελέσματα της παραπάνω υλοποίησης είναι τα εξής:

Cores	Execution Time
1	7.0054
2	3,6600
4	2.1916
8	1.2036
16	0.9116
32	0.4876
64	0.4838

Figure 4: Kmeans reduction - Execution Time for different number of cores



(a) Performance (Execution time in sec)

(b) Speedup

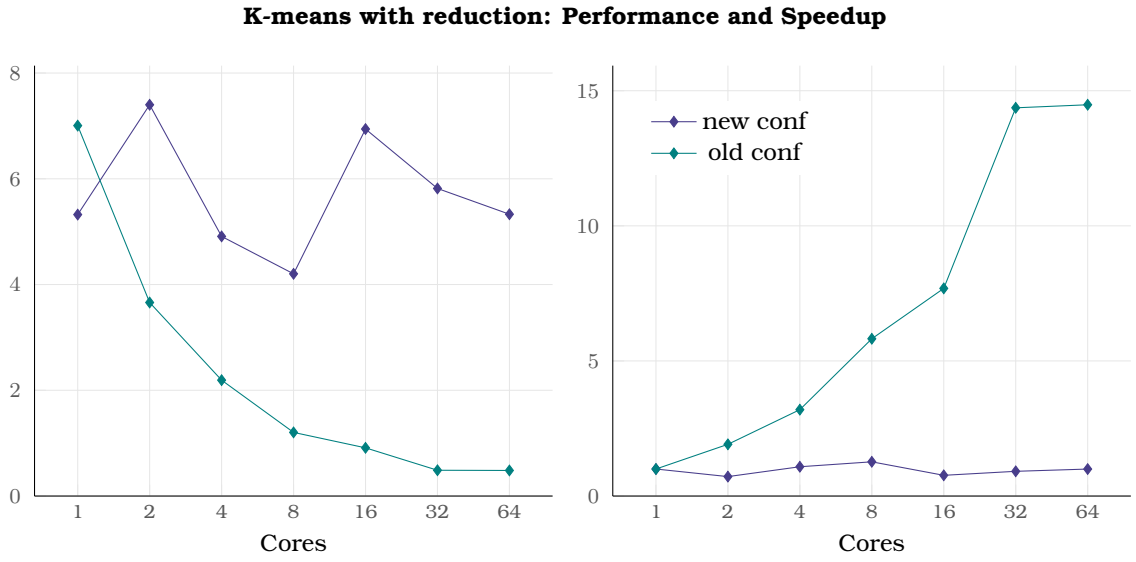
Όπως διακρίνουμε, με την υλοποίηση αυτή καταφέρνουμε πολύ καλύτερα αποτελέσματα και μάλιστα γραμμικό speedup (με εξαίρεση το speedup μεταξύ των 32 και των 64 νημάτων που παραμένει σταθερό). Αυτό είναι αναμενόμενο, δεδομένου ότι με την μέθοδο αυτή αποφεύγουμε την "σειριοποίηση" της κρίσιμης περιοχής (η οποία βασικά δεν υπάρχει αφού δεν υπάρχουν shared δεδομένα). Το μόνο σημαντικό κόστος που έχει είναι του reduction, το οποίο μάλιστα το έχουμε υλοποιήσει έτσι, ώστε να το εκτελεί ένα thread, ενώ έχει και το μειονέκτημα ότι απαιτεί σημαντικά περισσότερη μνήμη.

Για την περίπτωση του σταθερού speedup μεταξύ των 32 και των 64 νημάτων, το μηχάνημα του sandman έχει 32 φυσικούς πυρήνες με δυνατότητα αξιοποίησής τους σαν 64 μέσω hyperthreading, όπου 2 νήματα με διαφορετικό τύπο εντολής μπορούν να εκτελεστούν παράλληλα στον ίδιο φυσικό πυρήνα. Αν, αντιθέτως, τα νήματα που δρομολογούνται στον ίδιο φυσικό πυρήνα έχουν να εκτελέσουν ίδιο τύπο εντολών, όπως συμβαίνει στα παράλληλα threads, αυτά εκτελούνται σειριακά και το hyperthreading δεν φέρει καλύτερα αποτελέσματα.

Έστω configuration Size, Coords, Clusters, Loops = 256, 1, 4, 10. Τα αντίστοιχα αποτελέσματα και διαγράμματα επίδοσης της παραπάνω υλοποίησης για αυτό το configuration είναι τα εξής:

Cores	**Execution Time**
1	5.3227
2	7.3996
4	4.9103
8	4.2023
16	6.9408
32	5.8170
64	5.3296

Figure 5: Kmeans reduction with configuration {256, 1, 4, 10} - Execution Time for different number of cores



(a) Performance (Execution time in sec)

(b) Speedup

Από τα διαγράμματα διαπιστώνουμε ότι η παραπάνω υλοποίηση δεν είναι scalable. Συγκριτικά με το προηγούμενο configuration, Size, Coords, Clusters, Loops = 256, 16, 16, 10, στο οποίο παρατηρήσαμε γραμμικό speedup, όχι μόνο έχουμε χειρότερους χρόνους, παρότι πρόκειται για μικρότερο configuration, αλλά δεν έχουμε και την ίδια (γραμμική) συμπεριφορά.
Τα παραπάνω οφείλονται στο first-touch σε συνδυασμό με τα φαινομένα false-sharing. Το first-touch είναι η πολιτική του ΛΣ σε NUMA αρχιτεκτονικές κατά την οποία το memory page, όταν γραφτεί πρώτη φορά από έναν νήμα, θα γίνει allocated στην cache του επεξεργαστή στον οποίο έτρεξε και κάθε φορά που οποιοδήποτε άλλο νήμα χρειαστεί αυτό το page - που λόγω false-sharing αν και το κάθε νήμα θα χρειάζεται άλλα δεδομένα, θα χρειάζεται το ίδιο page - δεν θα μπορεί να το μεταφέρει στην cache του, άλλα θα πρέπει να το βρίσκει σε εκείνη την cache.

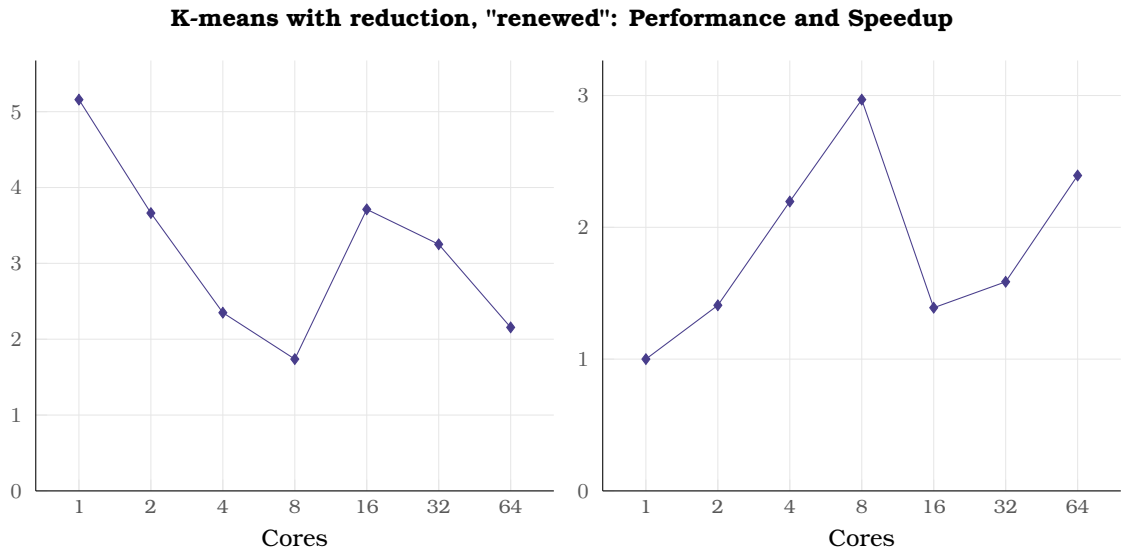
Έτσι, στο πρόβλημά μας κατά την πραγματική τοποθέτηση των local_newClusters δεδομένων στη μνήμη (που γίνεται σειριακά και συνεχόμενα, αφού οι memory allocators προσπαθούν να αποφύγουν τον κατακερματισμό της μνήμης) επειδή τα δεδομένα αυτά είναι λίγα στο νέο configuration η cache ενός node μπορεί να τα αποθηκεύσει όλα. Αυτό κοστίζει ιδιαίτερα όταν, για πολλά νήματα, θα χρειαστεί επικοινωνία μεταξύ επεξεργαστών που βρίσκονται σε διαφορετικά NUMA nodes, κάτι που δικαιολογεί το "άλμα" στον χρόνο εκτέλεσης μετά τα 8 νήματα, αφού ο sandman έχει nodes 8 πυρήνων και, συνεπώς, μέχρι και τους 8 πυρήνες χρησιμοποιούνταν 1 node. Αντίστοιχα, δικαιολογείται και η αύξηση του χρόνου εκτέλεσης στα 32 νήματα, ενώ από τα 32 στα 64 έχουμε "την αποτυχία" του hyperthreading. Αντιθέτως, με το παλιό configuration, που, λόγω των πολλών δεδομένων, δεν ήταν δυνατόν να γίνουν allocate όλα τα δεδομένα στις cache ενός node, μοιράζονται στα επιμέρους nodes με αποτέλεσμα να έχουμε λιγότερες καθυστερησεις.

Για την αντιμετώπιση του παραπάνω παραλληλοποιήσαμε την calloc. Με αυτόν τον τρόπο, δεδομένου και του "δεσίματος" των νημάτων σε συγκεκριμένο επεξεργαστή (affinity), θα μοιραστεί η local_newClusters στις διάφορες caches, και όχι μόνο στου ενός node, και θα ελλαττώσουμε τις καθυστερήσεις των νημάτων των υπόλοιπων nodes.

```
    /*
     * Hint for false-sharing
     * This is noticed when numCoords is low (and neighboring local_newClusters exist
    close to each other).
     * Allocate local cluster data with a "first-touch" policy.
     */
    // Initialize local (per-thread) arrays (and later collect result on global arrays)
   #pragma omp parallel for private(k)
    for (k=0; k<nthreads; k++)
    {
        local_newClusterSize[k] = (typeof(*local_newClusterSize)) calloc(numClusters,
    sizeof(**local_newClusterSize));
        local_newClusters[k] = (typeof(*local_newClusters)) calloc(numClusters *
    numCoords, sizeof(**local_newClusters));
    }
```

Listing 4: False-sharing correction code snippet

Τα αποτελέσματα της υλοποίησης με την παραπάνω "διόρθωση" είναι τα εξής:



(c) Performance (Execution time in sec)

(d) Speedup

Αμοιβαίος αποκλεισμός - Κλειδώματα στον K-means

Cores	nosync_lock	pthread_mutex_lock	pthread_spin_lock	tas_lock	ttas_lock	array_lock	clh_lock	pragma_omp_critical
1	0.9959	0.9442	1.0224	1.0113	1.0216	1.0709	1.0174	1.0242
2	0.6030	0.7844	0.6636	0.6697	0.6707	0.6752	0.6723	0.7286
4	0.3614	0.7760	0.7341	0.6930	0.6986	0.5335	0.5017	0.6216
8	0.2887	1.1715	0.9521	1.2048	0.9158	0.5738	0.5196	0.8733
16	0.7271	2.8434	2.1707	4.6646	2.1230	1.4030	1.1377	4.3255
32	1.1632	3.8247	4.4040	9.8164	4.4573	2.3027	1.7519	8.1650
64	0.9735	4.1243	5.8494	9.6703	6.9949	1.9873	1.2688	6.1597

Figure 6: Kmeans - Execution Time for different locks and number of cores

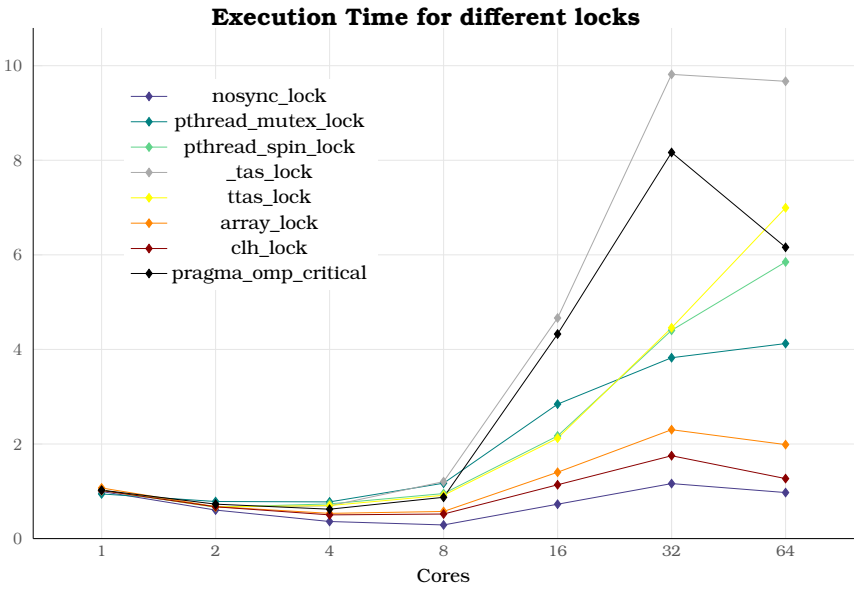


Figure 7: K-means - Execution Time using different locks

nosync lock Πρόκειται για την υλοποίηση της κρίσιμης περιοχής χωρίς να πραγματοποιείται συγχρονισμός. Όπως αναφέρεται και στην εκφώνηση, ενώ, σαφώς, δεν παράγει σωστά αποτελέσματα, το χρησιμοποιούμε ως το κάτω όριο χρόνου εκτέλεσης που μπορεί να επιτευχθεί.

pthread mutex lock Πρόκειται για το κλείδωμα mutex της βιβλιοθήκης pthread. Η υλοποίηση του βασίζεται στην χρήση μίας κοινής μεταβλητής-"αντικειμένου" (lock->mutex) που χρησιμοποιείται για τη συχρονισμένη πρόσβαση πολλών νημάτων στο κρίσιμο κομμάτι του προγράμματος. Εάν το mutex είναι διαθέσιμο, όποιο νήμα θελήσει να μπει στην κρίσιμη περιοχή το "παίρνει" και μπαίνει, ενώ τα υπόλοιπα νήματα που θα προσπαθήσουν να μπουν θα γίνονται blocked (δεν χρησιμοποιοεί ενεργά την CPU και το λειτουργικό μπορεί να την παραχωρήσει σε άλλη διεργασία μέχρι να γίνει unblocked). Όταν το νήμα που εκτελεί το κρίσιμο τμήμα τελειώσει, εκτελεί την συνάρτηση pthread_mutex_unlock η οποία όχι μόνο απελευθερώνει το mutex αλλα είναι και υπεύθυνη για το unblocking ενός απο τα νήματα που περιμένουν να αποκτήσουν πρόσβαση στο τμήμα.

```
#include <pthread.h>
#include "alloc.h"
#include "lock.h"
char LOCKNAME[32];

struct lock_struct {
        pthread_mutex_t mutex;
};

lock_t *lock_init(int nthreads)
{
        strcpy(LOCKNAME,"pthread-mutex");
        lock_t *lock;

        XMALLOC(lock, 1);
        pthread_mutex_init(&lock->mutex, NULL);
        return lock;
}

void lock_free(lock_t *lock)
{
        pthread_mutex_destroy(&lock->mutex);
        XFREE(lock);
}

void lock_acquire(lock_t *lock)
{
        lock_t *l = lock;

        pthread_mutex_lock(&lock->mutex);
}

void lock_release(lock_t *lock)
{
        lock_t *l = lock;
        pthread_mutex_unlock(&lock->mutex);
}
```

Listing 5: Pthread mutex lock

pthread spin lock Πρόκειται επίσης για κλείδωμα της βιβλιοθήκης pthread. Η διαφορά με το προηγούμενο κλείδωμα έγκειται στο γεγονός ότι τα νήματα που θέλουν να εκτελέσουν το κρίσιμο τμήμα, ενώ το κλειδί δεν είναι διαθέσιμο (κάποιο νήμα εκτελεί εκείνο τον κώδικα της κρίσιμης περιοχής), δεν περιμένουν blocked, αλλά κάνοντας busy wait, δηλαδή ελέχουν συνεχώς αν είναι διαθέσιμο το κλείδωμα μέχρι να το αποκτήσουν. Αυτό μπορεί να φέρει καλύτερα αποτελέσματα για χαμηλό αριθμό από νήματα, όπως παρατηρούμε για λιγότερα από 32 νήματα, καθώς το busy wait καθιστά πιο γρήγορη την διεκδίκηση του lock, συγκριτικά με την επικοινωνία (notify-unblock) που συμβαίνει στο mutex. Ωστόσο, η υλοποίηση αυτή φέρει και 2 μειονεκτήματα: Το πρώτο είναι ότι όταν ο αριθμός των νημάτων αυξάνει σημαντικά όπως στην περίπτωση μας για 32 και 64 νήματα ο χρόνος εκτέλεσης της υλοποιήσης με spin lock είναι αρκετά μεγαλύτερος σε σχέση τον χρόνο του mutex.Αυτό συμβαίνει καθώς η αύξηση του πλήθους των νημάτων οδηγεί σε μεγαλύτερο ανταγωνισμό ανάμεσα στα νήματα που διεδικούν το lock οπότε και μεγαλύτερη καθυστέρηση σε αντίθεση με τα mutex που βάζουν τα νήματα σε κατάσταση block και τα ειδοποιούν όταν το κρίσιμο τμήμα είναι ξανά προσβάσιμο και συνεπώς υπάρχει χρονοδρομολόγηση. Επιπλέον, οι CPU με το busy wait αποτρέπουν νήματα που εκτελούν άλλες διεργασίες ή άλλα τμήματα (μη κρίσιμα) του κώδικα να μπλοκάρουν από άλλα που είναι σε busy wait στην ίδια CPU (αν και αυτό δεν επηρεάζει στον κώδικα του kmeans καθώς έχουμε μία CPU ανά νήμα με affinity)

```
 #include <pthread.h>

#include "alloc.h"
#include "lock.h"
char LOCKNAME[32];

struct lock_struct {
        pthread_spinlock_t spinlock;
};

lock_t *lock_init(int nthreads)
{
        strcpy(LOCKNAME,"pthread-spinlock");
        lock_t *lock;

        XMALLOC(lock, 1);
        pthread_spin_init(&lock->spinlock, PTHREAD_PROCESS_SHARED);
        return lock;
}

void lock_free(lock_t *lock)
{
        pthread_spin_destroy(&lock->spinlock);
        XFREE(lock);
}

void lock_acquire(lock_t *lock)
{
        lock_t *l = lock;

        pthread_spin_lock(&lock->spinlock);
}

void lock_release(lock_t *lock)
{
```

```
        lock_t *l = lock;
        pthread_spin_unlock(&lock->spinlock);
}
```

Listing 6: Pthread mutex lock

tas lock Πρόκειται για ένα κλείδωμα το οποίο βασίζεται σε ατομικές εντολές. Τα νήματα έχουν μια κοινή μεταβλητή-θέση μνήμης η οποία καθορίζει το αν η κρίσιμη περιοχή είναι διαθέσιμη και κάθε ενδιαφερόμενο για την κρίσιμη περιοχή νήμα εκτελεί ατομικά το διάβασμα (read) της κοινής θέσης μνήμης , ανεξάρτητα από την τιμή της (0 ή 1), και την εγγραφή(write) της τιμής 1 σε αυτήν.Στην συνέχεια ελέγχει την τιμή που διάβασε και αν αυτή είναι 0 τότε μπαίνει στο κρίσιμο τμήμα αλλιώς ξαναπροσπαθεί.Προφανώς όταν ένα νήμα βγαίνει από την κρίσιμη περιοχή γράφει ατομικά στην κοινή θέση μνήμης την τιμή 0.Η συγκεκριμένη υλοποίηση καταλήγει να έχει τον χειρότερο χρόνο εκτέλεσης (συγκριτικά με τα υπόλοιπα κλειδώματα) και αυτό οφείλεται στο cache coherence πρωτόκολλο, αφού το κάθε νήμα γράφοντας στην κοινή θέση μνήμης (ενώ τις περισσότερες φορές είναι μη απαραίτητο γράφει ξανά την τιμή 1) κάνει invalid την αντίστοιχη cache line με αποτέλεσμα να γίνεται περιττή (υπερβολική) χρήση του διαδρόμου. (Αξίζει να σχολιάσουε ότι για 32 και 64 threads έχουμε ίδιο χρόνο εκτέλεσης και όχι χειρότερο διότι στο sandman οι επιπλέο 32 νήματα προέρχονται απο hyperthreading οπότε ο αριθμός των caches , άρα και των invalidations είναι 32 όπως και πρίν με αποτέλεσμα να έχουμε την ίδια περριτή κίνηση στο bus.)

```
#include "alloc.h"
#include "lock.h"
char LOCKNAME[32];

typedef enum {
        UNLOCKED = 0,
        LOCKED
} lock_state_t;

struct lock_struct {
        lock_state_t state;
};

lock_t *lock_init(int nthreads)
{
        strcpy(LOCKNAME,"tas");
        lock_t *lock;

        XMALLOC(lock, 1);
        lock->state = UNLOCKED;
        return lock;
}

void lock_free(lock_t *lock)
{
        XFREE(lock);
}

void lock_acquire(lock_t *lock)
{
        lock_t *l = lock;
```

```
        while (__sync_lock_test_and_set(&l->state, LOCKED) == LOCKED)
                /* do nothing */ ;
}

void lock_release(lock_t *lock)
{
        lock_t *l = lock;

        __sync_lock_release(&l->state);
}
```

Listing 7: tas lock

ttas lock Το ttas είναι μια βελτιωμένη έκδοση του tas προκειμένου να επιλύσει την άσκοπη χρήση του bus λόφο του cache coherence.Συγκεκριμένα κάθε thread διαβάζει ατομικά διαρκώς την κοινή θέση μνήμης μέχρι να δει τιμή 0 και μόνο τότε θα εκτελέσει την test and set "διαδικασία". Έτσι, αφού κάνουμε επίμονα reads δεν κάνουμε invalid την cache line των άλλων threads παρα μόνο όταν αυτό είναι απαραίτητο (στο κομμάτι του tas) και αποφεύγουμε την καθυστέρηση στον διάδρομο.Όντως παρατηρούμε μεγάλη βελτίωση στου χρόνους σε σχέση με το tas ,αλλά και πάλι βλέπουμε ότι όσο αυξάνονται τα νήματα ο χρόνο εκτέλεσης αυξάνεται σημαντικά γιατί έχουμε απο περσσότερα νήματα διεκδίκηση του ποιός θα εκτελέσει πρώτος ατομική εντολή (atomic operations έχουν κόστος).

```
#include "alloc.h"
#include "lock.h"
char LOCKNAME[32];

typedef enum {
        UNLOCKED = 0,
        LOCKED
} lock_state_t;

struct lock_struct {
        volatile lock_state_t state;
};

lock_t *lock_init(int nthreads)
{
        strcpy(LOCKNAME,"ttas");
        lock_t *lock;

        XMALLOC(lock, 1);
        lock->state = UNLOCKED;
        return lock;
}

void lock_free(lock_t *lock)
{
        XFREE(lock);
}

void lock_acquire(lock_t *lock)
{
        lock_t *l = lock;
```

```
        do {
                while (l->state == LOCKED)
                        /* do nothing */ ;
        } while (__sync_lock_test_and_set(&l->state, LOCKED) == LOCKED);
}

void lock_release(lock_t *lock)
{
        lock_t *l = lock;

        __sync_lock_release(&l->state);
}
```

Listing 8: ttas lock

array lock Η συγκεκριμένη υλοποίηση λύνει το πρόβλημα του ανταγωνισμού στην κοινή θέση μνήμης και αυτό επιτυγχάνεται με το να έχουμε έναν πίνακα ώστε το κάθε νήμα να έχει μια "private" θέση μνήμης.Συγκεκριμένα αρχικοποιούμε έναν πίνακα,(ανάλογου μήκους με τον αριθμό των νημάτων) με FALSE εκτός από την πρώτη θέση και μια κοινή μεταβλητή(tail) αρχικοποιημένη στο 0.Το κάθε νήμα που θέλει να μπει στην κρίσιμη περιοχή διαβάζει και αυξάνει την τιμή του tail(ατομικά) η οποία του δεικτοδοτεί ποια θεση του πίνακα του ανήκει(slot), απο όπου διαβάζει συνεχόμενα την τιμή (busy wait) μέχρι να διαβάσει true και να μπει στην κρίσιμη περιοχή .Βγαίνοντας από την κρίσιμη περιοχή θέτει την τιμή(slot) σε false και ορίζει το επόμενο slot true για να έχει πρόσβαση στην κρίσιμη περιοχή το επόμενο νήμα. Με αυτήν την υλοποίηση εκμεταλλευόμαστε το busy wait αποφεύγοντας ταυτόχρονα τα race conditions σε κοινή θέση μνήμης. Παρατήρηση : Χρησιμοποιείται modulo ανάλογο του nthreads για την επαναχρησιμοποίηση των slots από threads που ξαναδιεκδικούν την πρόσβαση στην κρίσιμη περιοχή. Στο διάγραμμα φαίνεται ότι οι χρόνοι εκτέλεσης ακολουθούν την ίδια μορφή με αυτούς της no-sync υλοποιήσης (ιδανική περίπτωση) οπότε καταλαβαίνουμε ότι η διαφορά οφείλεται μόνο στο overhaed της υλοποιήσης (αρχικοποιήσηης array , tail,...)

```
#include "alloc.h"
#include "lock.h"
char LOCKNAME[32];

#define FALSE 0
#define TRUE  1

#define FLAG_ENTRY_SIZE 1024

struct lock_struct {
        volatile char *flag;
        char padding1[64 - sizeof(char *)];
        unsigned long long tail;
        char padding2[64 - sizeof(unsigned long long)];
        int size;
};

lock_t *lock_init(int nthreads)
{
        strcpy(LOCKNAME,"array-based");
        lock_t *lock;
        int i;
```

```
        XMALLOC(lock, 1);

        lock->size = nthreads * FLAG_ENTRY_SIZE;
        lock->tail = 0;
        XMALLOC(lock->flag, lock->size);

        for (i=0; i < lock->size; i++)
                lock->flag[i] = FALSE;
        lock->flag[0] = TRUE;

        // printf("lock->size = %d\n", lock->size);
        return lock;
}

void lock_free(lock_t *lock)
{
        lock_t *l = lock;
        XFREE((void*)l->flag);
        XFREE(l);
}

__thread int mySlot;

void lock_acquire(lock_t *lock)
{
        lock_t *l = lock;
        int slot = __sync_fetch_and_add(&l->tail, FLAG_ENTRY_SIZE) % l->size;

        mySlot = slot;
        // if (mySlot < 0) {
        // printf("mySlot = %d\n", mySlot);
        // printf("Tail = %d\n", l->tail);
        // }

        while (l->flag[slot] == FALSE)
                /* do nothing */ ;
}

void lock_release(lock_t *lock)
{
        lock_t *l = lock;
        int slot = mySlot;

        l->flag[slot] = FALSE;
        l->flag[(slot+FLAG_ENTRY_SIZE) % l->size] = TRUE;
}
```

Listing 9: array lock

clh lock Η υλοποίηση αυτή αποτελεί βελτίωση της προηγούμενης με το να χρησιμοποιεί συνδεδεμένη λίστα αντί πινάκων αφού αυτοί χρειάζονται επιπλέον χρόνο λόγω της αρχικοποίησης.Κάθε νήμα που θέλει πρόσβαση στην κρίσιμη περιοχή συνδέεται, με αρχικοποίηση του πεδίου locked σε TRUE, ως νέος κόμβος στην λίστα με την συνάρτηση ατομικών εντολών __sync_lock_test_and_set(&l->tail, myNode) για να διασφαλίσουμε ότι δεν θα συνδεθούν δύο κόμβοι ταυτόχρονα σε έναν προηγούμενο(το πρώτο νήμα συνδέεται σε έναν, αρχικοποιημένο σε FALSE, node-root).Στην συνέχεια κάθε κόμβος διαβάζει επίμονα την τιμή του πεδίου locked μέχρι αυτή να γίνει FALSE , οπότε και μπαίνει στο κρίσιμο τμήμα, ενώ βγαίνοντας προφανώς θέτει την δική του τιμή σε False για το επόμενο στη λίστα κομβο-νήμα.

```
#include "alloc.h"
#include "lock.h"
char LOCKNAME[32];

#define FALSE 0
#define TRUE  1

#define FLAG_ENTRY_SIZE 1

typedef struct {
        volatile char locked; /* FALSE or TRUE. */
//      char padding[63];
        char padding[1023];
} clh_node_t;
//} __attribute__ ((aligned(64))) clh_node_t;

struct lock_struct {
        clh_node_t *tail;
};

/**
 * These are GCC's magic. Per thread variables.
 * They are initialized to 0 ( = NULL).
 **/
__thread clh_node_t *myNode;
__thread clh_node_t *myPred;

lock_t *lock_init(int nthreads)
{
        strcpy(LOCKNAME,"clh-queue");
        lock_t *lock;
        clh_node_t *tail;
        int i;

        XMALLOC(lock, 1);
        XMALLOC(tail, 1);

        lock->tail = tail;
        lock->tail->locked = FALSE;

        return lock;
}
void lock_acquire(lock_t *lock)
{
        lock_t *l = lock;
```

```
        if (!myNode)
                XMALLOC(myNode, 1);

        myNode->locked = TRUE;
        myPred = __sync_lock_test_and_set(&l->tail, myNode);

        while (myPred->locked == TRUE)
                /* do nothing */ ;
}

void lock_release(lock_t *lock)
{
        myNode->locked = FALSE;
        myNode = myPred;
}
```

Listing 10: clh lock

Παραλληλοποίηση του αλγορίθμου Floyd-Warshall

Αρχικά, θα συγκρίνουμε τους χρόνους εκτέλεσης του FW για τον γραμμικό και τον αναδρομικό αλγόριθμο, όπου για τον δεύτερο θα δοκιμάσουμε block sizes 32, 64 και 128. Όπως βλέπουμε και απο τον παρακάτω πίνακα ο αναδρομικός αλγόριθμος είναι σημαντικά καλύτερος σε σχέση με τον γραμμικό και μάλιστα αυτό φαίνεται πιο έντονα για μεγαλύτερα sizes του προβλήματος. Επίσης, διακρίνουμε ότι στον αναδρομικό αλγόριθμο για όσο μεγαλύτερο το block size τοσο καλύτερος ο χρόνος εκτέλεσης που επιτυγχάνουμε.

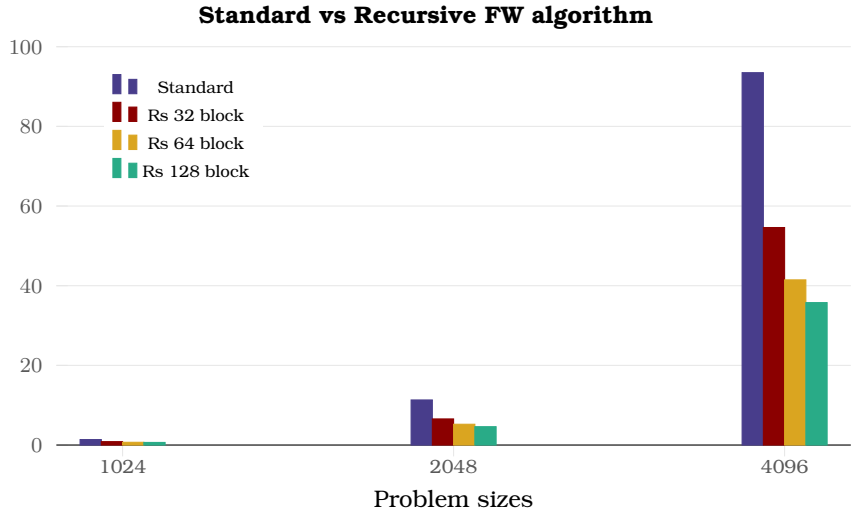


Figure 8: Execution time for Standard and Recursive (with different block sizes) algorithms

Στην συνέχεια σκοπός μας είναι να παραλληλοποιήσουμε τον αναδρομικό αλγόριθμο, ο οποίος, ωστόσο, δεν είναι ιδιαίτερα παραλληλοποιήσιμος, λόγω των εξαρτήσεων μεταξύ των περισσότερων εντολών (αναδρομών).
Αρχικά, στην main() μέσω της εντολής #pragma omp parallel επιτρέπουμε την παραλληλοποιήση μεσα στην αναδρομική συνάρτηση. Ωστόσο, χρειάζεται να καλέσουμε στη συνέχεια την εντολή #pragma omp single για να αναγκάσουμε τη συνάρτηση να τρέξει μόνο απο ένα νήμα, καθώς αυτή δεν είναι πλήρως παραλληλοποιήσιμη. Οπως φαίνεται και στον παρακάτω κώδικα μπορούμε να παραλληλοποιήσουμε μόνο τα βήματα 2,3 και τα 6,7, και για να τα το καταφέρουμε αυτό χρησιμοποιούμε τις εντολές #pragma omp task if (0) , #pragma omp taskwait και #pragma omp task. Μια πρώτη προσέγγιση θα ήταν το single thread, που καλείται στην main και εκτελεί τα "σειριακά" βήματα (1, 4, 5, 8), να καλεί #pragma omp task και για το βήμα 2 και για το βήμα 3 (αντίστοιχα για τα βήματα 6, 7), τα οποία έτσι θα εκτελούνται παράλληλα από 2 άλλα threads, και να τα περιμένει. Ωστόσο, αφού το single thread μετά το βήμα 1 δεν εκτελεί κάποιο κώδικα, θα ήταν προτιμότερο για λόγους locality των δεδομένων (γλυτώνουμε περιττή μετακίνηση των δεδομένων) να εκτελεί και ένα εκ των δύο επόμενων βημάτων (αφού αυτά έχουν εξάρτηση από το βήμα 1), βελτιώνοντας την επίδοση του κώδικα. Έτσι, καταλήγουμε στην εξής υλοποίηση:

```
main()
    #pragma omp parallel
        {
            #pragma omp single
            {
                FW_SR(A,0,0, A,0,0,A,0,0,N,B);
            }
        }
```

Listing 11: Parallel FW in main()

```
void FW_SR (int **A, int arow, int acol,
            int **B, int brow, int bcol,
            int **C, int crow, int ccol,
            int myN, int bsize)
{
        int k,i,j;

        if(myN<=bsize)
                for(k=0; k<myN; k++)
                        for(i=0; i<myN; i++)
                                for(j=0; j<myN; j++)
                                        A[arow+i][acol+j]=min(A[arow+i][acol+j], B[brow+i
    ][bcol+k]+C[crow+k][ccol+j]);
        else {
                FW_SR(A,arow, acol,B,brow, bcol,C,crow, ccol, myN/2, bsize);

                #pragma omp task if (0)
                {
                        #pragma omp task
                        FW_SR(A,arow, acol+myN/2,B,brow, bcol,C,crow, ccol+myN/2, myN/2,
    bsize);
                        FW_SR(A,arow+myN/2, acol,B,brow+myN/2, bcol,C,crow, ccol, myN/2,
    bsize);
                        #pragma omp taskwait
                }

                FW_SR(A,arow+myN/2, acol+myN/2,B,brow+myN/2, bcol,C,crow, ccol+myN/2, myN
    /2, bsize);
                FW_SR(A,arow+myN/2, acol+myN/2,B,brow+myN/2, bcol+myN/2,C,crow+myN/2,
    ccol+myN/2, myN/2, bsize);

                #pragma omp task if (0)
                {
                        #pragma omp task
                        FW_SR(A,arow+myN/2, acol,B,brow+myN/2, bcol+myN/2,C,crow+myN/2,
    ccol, myN/2, bsize);
                        FW_SR(A,arow, acol+myN/2,B,brow, bcol+myN/2,C,crow+myN/2, ccol+
    myN/2, myN/2, bsize);
                        #pragma omp taskwait
                }

                FW_SR(A,arow, acol,B,brow, bcol+myN/2,C,crow+myN/2, ccol, myN/2, bsize);
        }
}
```

Listing 12: Parallel FW in function

Τα αποτελέσματα από την παραπάνω υλοποίηση είναι τα εξής:

Cores	**Problem size**		
	1024	2048	4096
1	0.8545	6.5568	54.6060
2	0.5131	3.4362	29.4503
4	0.3176	2.0765	16.4986
8	0.2655	1.6212	11.3208
16	0.2970	1.7067	10.5930
32	0.7284	2.5641	17.4907
64	0.6123	6.2676	27.8241

Figure 9: Recursive FW with block size 32

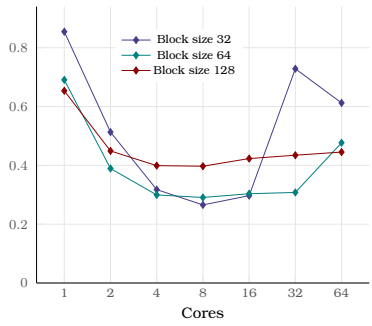
Cores	**Problem size**		
	1024	2048	4096
1	0.6907	5.2204	41.4621
2	0.3893	2.7594	21.3721
4	0.2995	1.8900	12.8681
8	0.2908	1.6437	10.0989
16	0.3035	1.6936	9.7865
32	0.3037	1.7307	10.0990
64	0.4767	2.7063	16.2478

Figure 10: Recursive FW with block size 64

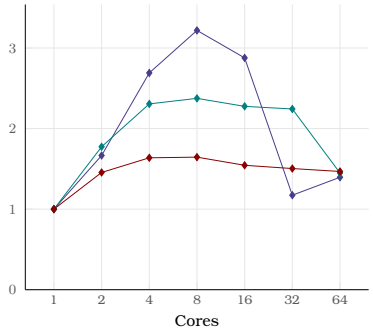
Cores	**Problem size**		
	1024	2048	4096
1	0.6534	4.6075	35.7598
2	0.4490	2.6520	18.9175
4	0.3991	2.2371	13.1763
8	0.3971	2.1260	11.9522
16	0.4231	2.2847	12.5821
32	0.4345	2.3775	12.9360
64	0.4452	2.5525	15.3858

Figure 11: Recursive FW with block size 128

FW: Performance and Speedup for problem size 1024

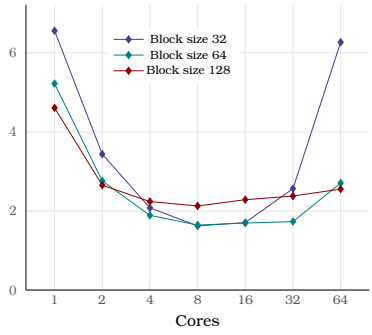


(a) Performance (Execution time in sec)

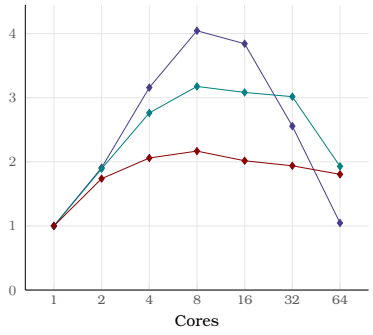


(b) Speedup

FW: Performance and Speedup for problem size 2048

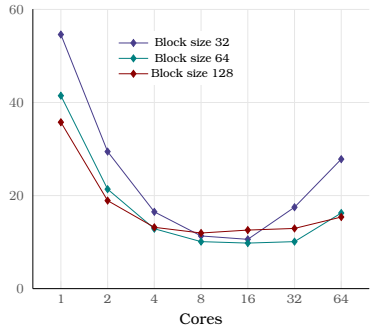


(c) Performance (Execution time in sec)

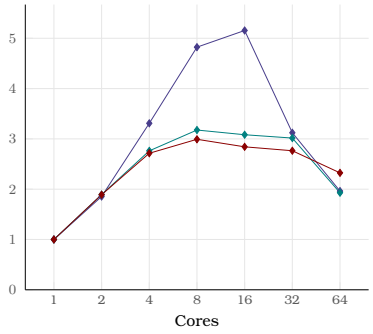


(d) Speedup

FW: Performance and Speedup for problem size 4096

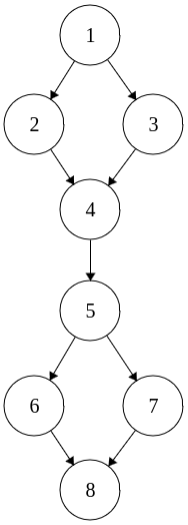


(e) Performance (Execution time in sec)



(f) Speedup

Από τα παραπάνω αποτελέσματα διαπιστώνουμε, αρχικά, ότι σε αντιπαράθεση με την σειριακή υλοποίηση, καλύτερη επίδοση έχουμε για block size 64 (και όχι 128). Αυτό οφείλεται στο ότι για block size 64 το μέγεθος του υποπροβλήματος είναι αρκετά μεγάλο ώστε να γίνονται λιγότεροι υπολογισμοί, συγκριτικά με μικρότερα block sizes, αλλά αρκετά μικρό, συγκριτικά με μεγαλύτερα block sizes, ώστε τα δεδομένα να χωρούν στην cache και να αποφευγόνται "περιττά" cache misses και οι καθυστερήσεις κατά την πρόσβαση στην κύρια μνήμη που αυτά επιφέρουν (data locality).
Ωστόσο, για όλα τα διάφορα block sizes παρατηρούμε ότι δεν επιτυγχάνουμε σημαντικό speedup. Αυτό είναι αναμενόμενο αφού πρόκειται για ένα memory bound πρόβλημα (μεγάλο μέρος του χρόνου εκτέλεσης οφείλεται στην πρόσβαση στη μνήμη, κάτι που δεν μπορούμε να αποφύγουμε, παρά μόνο να το βελτιώσουμε λίγο με την cache όπως σχολιάσαμε παραπάνω), καθώς και επειδή πρόκειται για ένα πρόβλημα (η συγκεκριμένη υλοποίηση τουλάχιστον) με μεγάλο μη-παραλληλοποιήσιμο μέρος (N. Amdahl: ο συνολικός χρόνος εκτέλεσης είναι bound από τον χρόνο εκτέλεσης του μη παραλληλοποιήσιμου κομματιού όσο καλη παραλληλοποίηση και να επιτύχουμε). Το τελευταίο διαπιστώνεται και από το παρακάτω task graph, όπου 1, 2, κτλ. οι αναδρομικές κλήσεις της συνάρτησης όπως καλούνται στο σώμα αυτής (τα αντίστοιχα βήματα όπως τα αναφέρουμε παραπάνω).



Ταυτόχρονες Δομές Δεδομένων

Στο σημείο αυτό θα μελετήσουμε τις εξής διάφορες ταυτόχρονες υλοποιήσεις μίας απλά συνδεδεμένης ταξινομημένης λίστας:

- Coarse-grain locking
- Fine-grain locking
- Optimistic synchronization
- Lazy synchronization
- Non-blocking synchronization

Αρχικά, παρατίθενται παρακάτω τα διαγράμματα Throughput - Cores για διαφορετικά μεγέθη λίστας (1024, 8192) και διαφορετικά ποσοστά λειτουργιών (100-0-0, 80-10-10, 20-40-40 και 0-50-50)

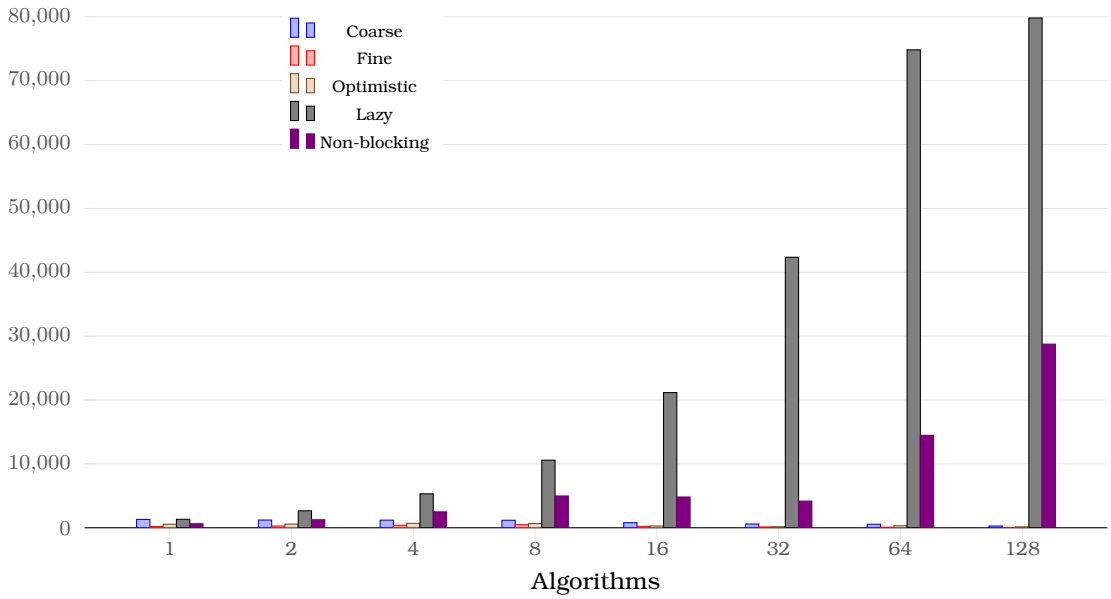


Figure 12: Throughput for configuration 100-0-0 and size 1024

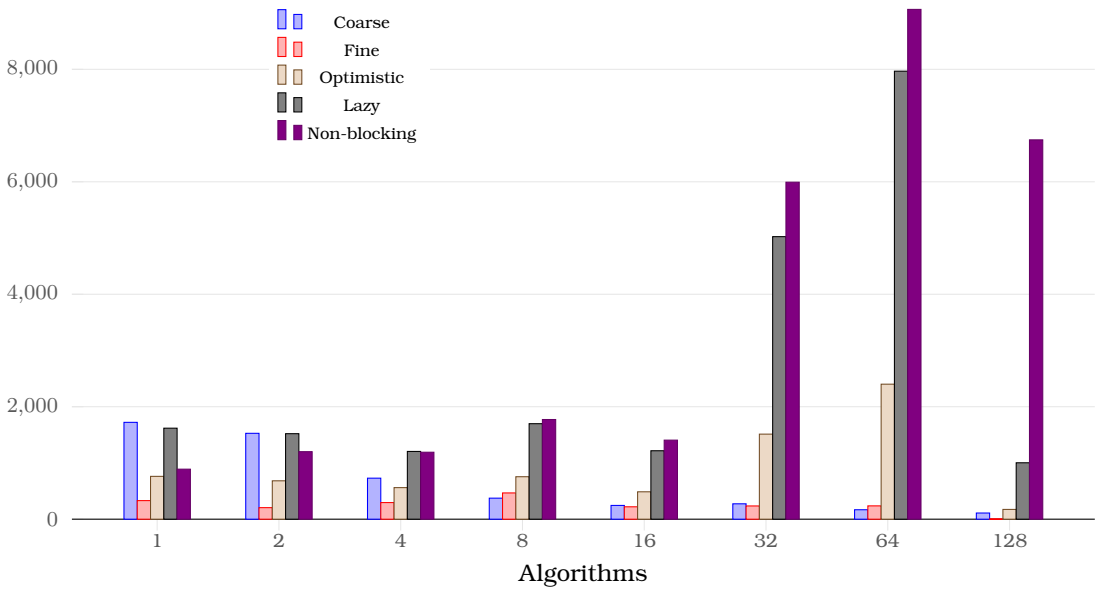


Figure 13: Throughput for configuration 80-10-10 and size 1024

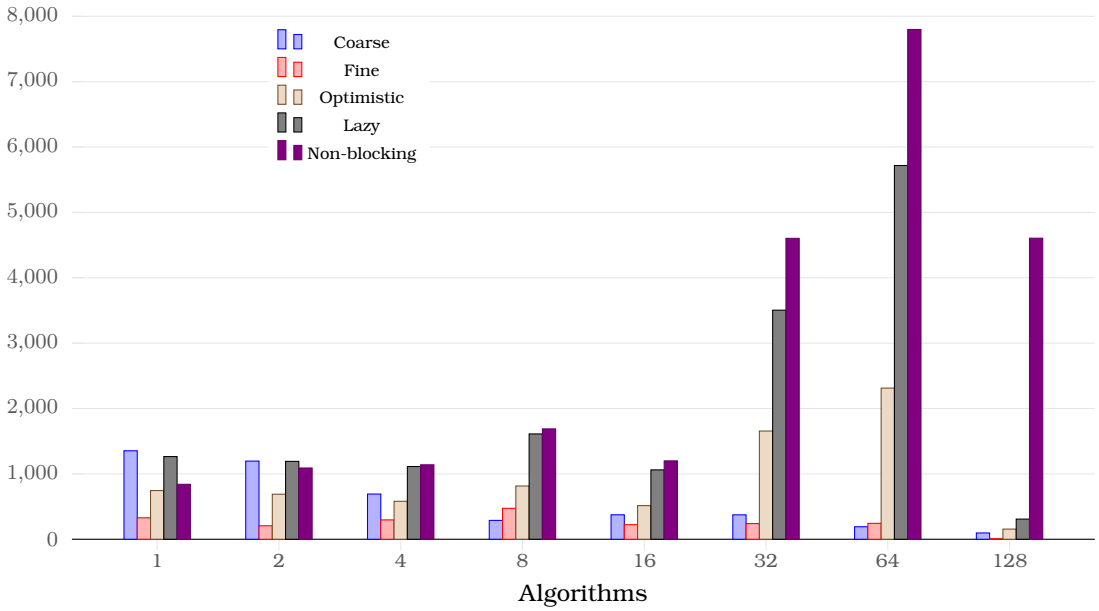


Figure 14: Throughput for configuration 20-40-40 and size 1024

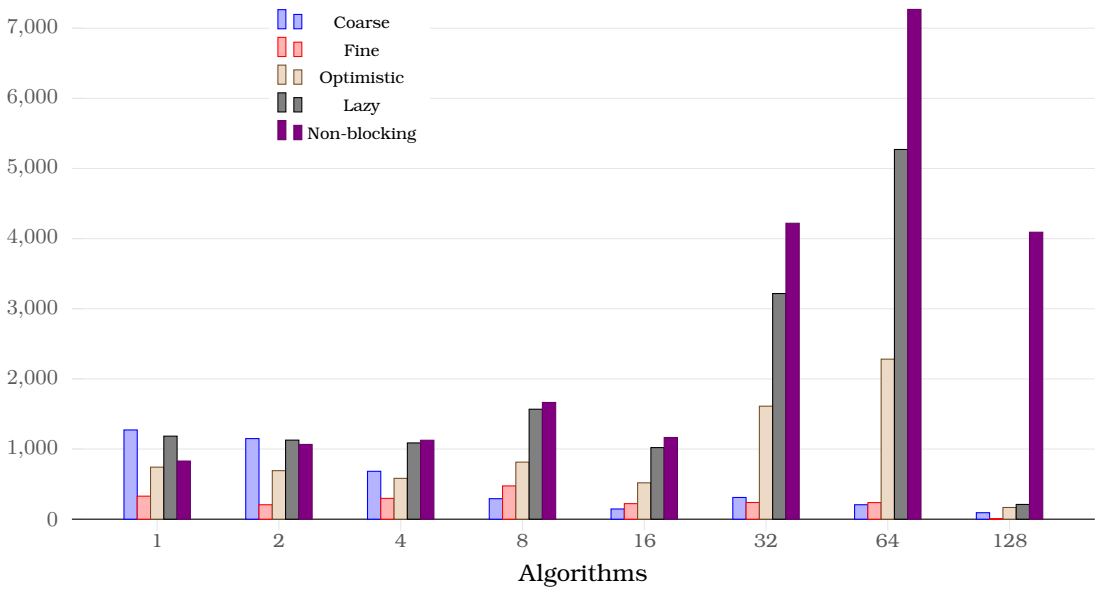


Figure 15: Throughput for configuration 0-50-50 and size 1024

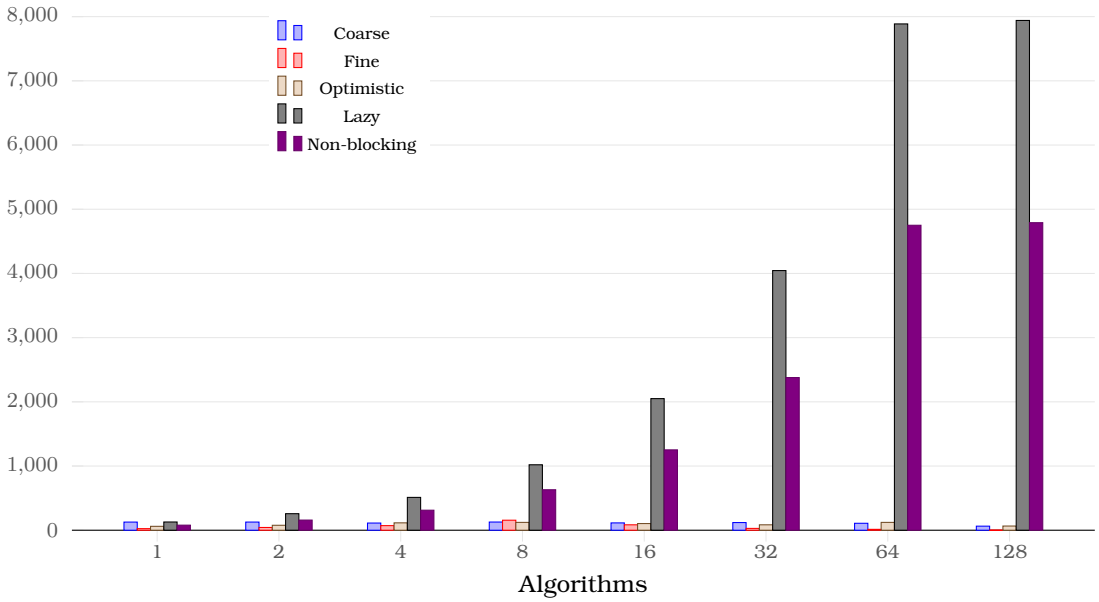


Figure 16: Throughput for configuration 100-0-0 and size 8192

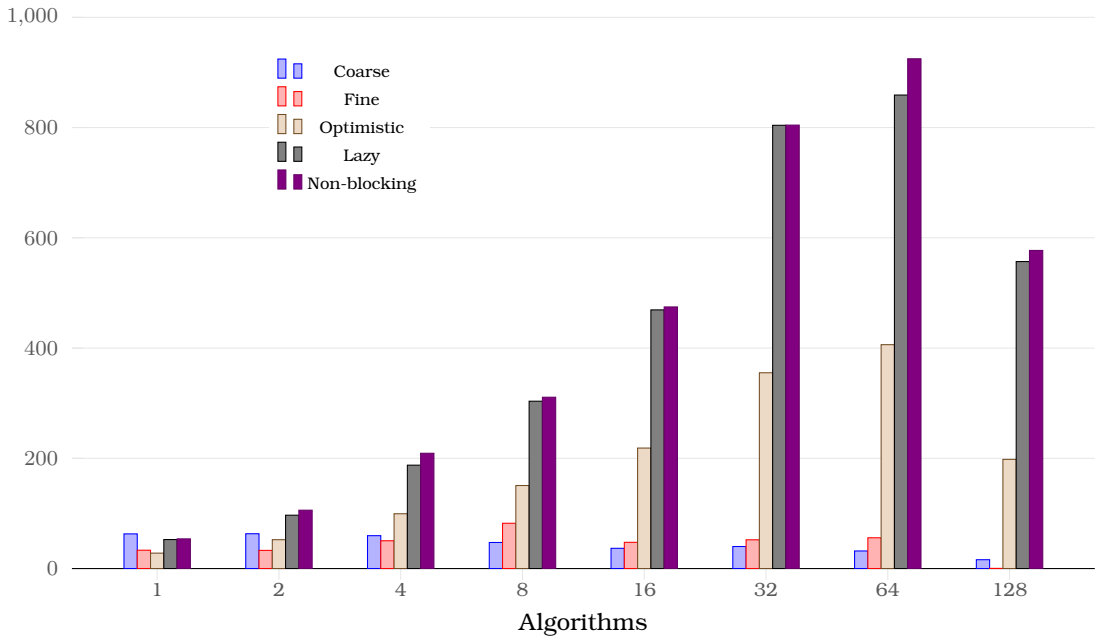


Figure 17: Throughput for configuration 80-10-10 and size 8192

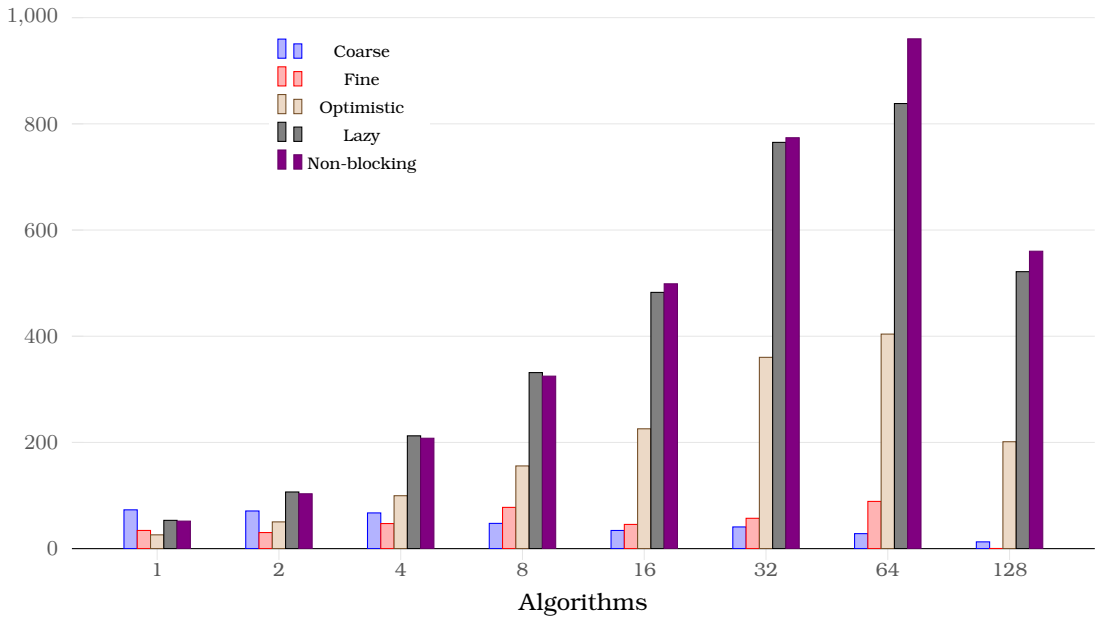


Figure 18: Throughput for configuration 20-40-40 and size 8192

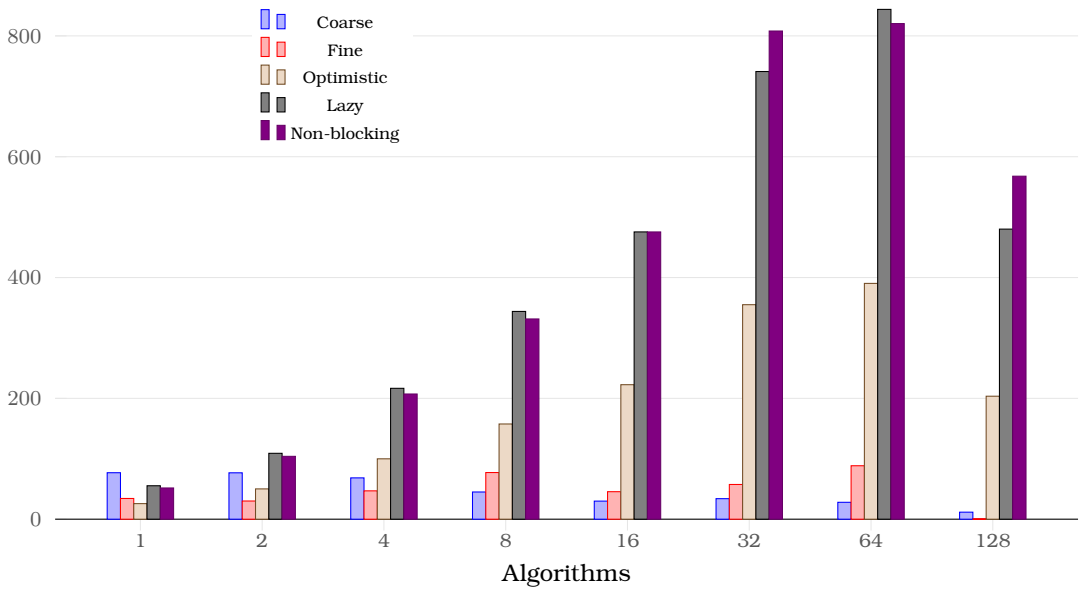


Figure 19: Throughput for configuration 0-50-50 and size 8192

Coarse-grain locking:
Στην υλοποίηση αυτή χρησιμοποιείται σε όλες τις λειτουργίες (add,remove,contain) ένα spin lock κλειδώνοντας όλη τη δομή και, συνεπώς, μόνο ένα νήμα κάθε φορά μπορεί να εκτελεί κάποια λειτουργία, ενώ όλα τα άλλα χρειάζεται να περιμένουν μέχρι αυτό να τελειώσει και να κάνει unlock. Πρόκειται για μια απλή προγραμματιστικά υλοποίηση, η οποία ωστόσο δεν είναι κλιμακώσιμη και φέρνει μη επιθυμητά αποτελέσματα (throughput), όπως φαίνεται και από τα παραπάνω γραφήματα, αφού με το κλείδωμα ολόκληρης της δομής οι λειτουργίες σειριοποιούνται.

Fine-grain locking:
Με την υλοποίηση αυτή σκοπεύουμε να αποφύγουμε το κλείδωμα ολόκληρης της δομής. Γνωρίζουμε ότι συγχρονισμός είναι απαραίτητος μόνο στην περιοχή της δομής που κοντά της εκτελείται μια λειτουργία (add,remove,contains), η οποία περιοχή "διαφέρει" ανάλογα με την λειτουργία: Για την αναζήτηση ενός στοιχείου χρειάζεται να έχουμε κλειδωμένους δύο κόμβους, τον τρέχοντα και τον επόμενο κόμβο που “κοιταμε” (“hand-over-hand” locking), για την εισαγωγή στοιχείου χρειάζεται να κλειδώσουμε μόνο τον προηγούμενο κόμβο (άλλα λόγω της αναζήτησης θα έχουμε κλειδωμένο και το επόμενο), ενώ για την διαγραφή, τον προς διαγραφή κόμβο και τον προηγούμενο του.
Η υλοποίηση αυτή παρατηρούμε, αρχικά, ότι μέχρι έναν αριθμό νημάτων εμφανίζει speedup (μεγαλύτερο throughput με αύξηση του αριθμού των νημάτων), κάτι αναμενόμενο δεδομένου ότι έτσι οι λειτουργίες μπορούν να παραλληλήθούν. Ωστόσο, φέρει δύο μειονεκτήματα: α) χρησιμοποιούνται αναλογικά πολλά locks ως προς τα νήματα και β) το hand-over-hand locking μας περιορίζει σε διάσχιση προς μια κατεύθυνση και από την αρχή της λίστας, καθώς σε άλλη περίπτωση υπάρχει η πιθανότητα να έχουμε deadlocks. Το πρώτο μειονέκτημα, πέρα από το επιπλέον overhead που συνεπάγεται, σημαίνει

ότι η δομή από κάποιον αριθμό νημάτων και μετά "γεμίζει" από κλειδωμένα nodes και οι λειτουργίες πλέον "σειριοποιούνται", κάτι που διαπιστώνεται και από τα γραφήματα για περισσότερα από 8 νήματα.

Optimistic: Με αυτή την υλοποίηση κάνουμε την αναζήτησή του στοιχείου που θέλουμε να εισάγουμε διαγράψουμε ή και διαβάσουμε (find) χωρίς κλειδώματα, αλλά μόλις το βρούμε κλειδώνουμε τους ανάλογους (σύμφωνα με τη λειτουργία) κόμβους και ελέγχουμε αν η δομή είναι συνεπής (αν ο προηγούμενος κόμβος ανήκει στην λίστα, συγκεκριμένα αν είναι προσβάσιμος από τον κόμβο-ρίζα, και αν ο επόμενος αυτού είναι ο τρέχων κόμβος. Αν αυτά δεν ικανοποιούνται, γίνονται released τα κλειδώματα και ολόκληρη η διαδικασία ξεκινάει από την αρχή.
Η υλοποίηση αυτή αποτελεί βελτίωση των προηγουμένων μόνο εάν δεν συμβαίνουν συχνά ασυνέπειες και εάν το κόστος της επανάληψης της όλης διαδικασίας, για τις όσες ασυνέπειες συμβούν, είναι μικρότερο από το κόστος του hand-on-hand traversal. Όταν αυτά ικανοποιούνται μπορούμε να έχουμε καλύτερο συγκριτικά με τις προηγούμενες υλοποιήσεις throughput και κλιμάκωση, αφού μπορεί να υπάρχει αρκετή παραλληλοποίηση των λειτουργιών. Επιπλέον, αξίζει να σχολιαστεί ότι για 64 νήματα (hyperthreading) η υλοποίηση αυτή φέρει καλύτερα αποτελέσματα, τόσο συγκριτικά με τις προηγούμενες υλοποιήσεις όσο και σε σχέση με τους υπόλοιπους αριθμούς νημάτων, εκτός από την περίπτωση των ποσοστών λειτουργιών 100-0-0). Αυτό οφείλεται στο ότι το hyperthreading μπορεί να είναι αποτελεσματικό μόνο αν έχουμε ετερογενείς εντολές, κάτι που δεν συμβαίνει μόνο στην περίπτωση 100-0-0. Ωστόσο, για 128 νήματα (4 νήματα-λογικοί πυρήνες ανά φυσικό) εξακολουθεί και αυτή η υλοποίηση να μην φέρει καλά αποτελέσματα (χαμηλό throughput).

Lazy: Η optimistic υλοποίηση παρουσιάζει και αυτή κάποια σημεία που μπορούν να βελτιωθούν. Αρχικά, το validation ενός κόμβου της λίστας είναι ιδιαίτερα χρονοβόρο, διότι πρέπει να ελεγχθεί ότι ο προηγούμενος κόμβος ανήκει σε αυτήν (ελέγχοντας-διατρέχοντας αναγκαστικά όλους τους προηγούμενους κόμβους μέχρι την ρίζα). Επιπλέον, η μέθοδος contains χρησιμοποιεί κλειδώματα, κάτι που μπορούμε με την παρακάτω υλοποίηση να αποφύγουμε:
Στην lazy υλοποίηση εκμεταλλευόμαστε ότι έχουμε επιπλέον μνήμη στον κάθε κόμβο ώστε να προσθέσουμε μια boolean μεταβλητή που να δηλώνει αν ο κόμβος ανήκει ή οχι στην λίστα (“λογική” διαγραφή, ενώ θα έπεται η φυσική διαγραφή με την οποία ο προηγούμενος κόμβος δείχνει στον επόμενο από τον προς διαγραφή κόμβο και όχι σε αυτόν). Με αυτόν τον τρόπο γλιτώνουμε το προαναφερθέντα χρονοβόρο έλεγχο του validate, αφού αν έχει αφαιρεθεί μια υπολίστα που συμπεριλαμβάνει τον κόμβο μας αυτός όπως και οι προηγούμενοι κόμβοι θα έχουν “μαρκαριστεί” κατάλληλα, ενώ ακόμα καθίσταται μη απαραίτητη η χρήση κλειδώματος στην contains, αφού δεν γίνεται κάποιος κόμβος να βρίσκεται στη λίστα και να έχει “μαρκαριστεί” ως deleted και το αντίστροφο (λόγω των κλειδωμάτων στην remove).
Η υλοποίηση αυτή εμφανίζει καλύτερο throughput από όλες τις προηγούμενες υλοποιήσεις και σε όλους τους συνδυασμούς λειτουργιών, καθώς και speedup, ιδιαίτερα στην περίπτωση που έχουμε μόνο contains, στην οποία περίπτωση πλέον χρειάζεται μόνο να βρει το στοιχείο και αν το βρει να ελέγξει την boolean μεταβλητή. Επίσης, παρατηρούμε ότι έχουμε καλά αποτελέσματα και για την περίπτωση των 128 νημάτων.

Non-blocking: Στην τελευταία αυτή υλοποίηση σκοπός είναι να απαλείψουμε πλήρως τα κλειδώματα απο τις μεθόδους contains,add και remove, το οποίο το καταφέρνουμε χρησιμοποιώντας ατομικές δομές. Συγκεκριμένα ενώνουμε στο πεδίο της διεύθυνσης του επόμενου που έχει ένας κόμβος και ένα bit που δείχνει αν ο τρέχον κόμβος ανήκει στην λίστα ή οχι (ένα λιγότερο bit στη διεύθυνση). Έτσι, ο έλεγχος της διεύθυνση και του validate bit μπορούν να γίνουν “ατομικά” (ένα xor στην ίδια ατομική δομή). Όπως και πριν, κάθε add /remove αφαιρεί πρώτα “λογικά” τον κόμβο (γράφει το "validation" bit σε ‘1’) και στη συνέχεια δοκιμάζει μία φορά να τον αφαιρέσει και “φυσικά” (αλλάζοντας την διεύθυνση του προηγούμενου). Αν δεν καταφέρει να τον αφαιρέσει/προσθέσει “φυσικά” αυτό θα γίνει κάποια άλλη στιγμή από κάποιο νήμα το οποίο κατά την διάσχιση της λίστας θα συναντήσει bit 1 (σε όσους λογικά σβησμενους κόμβους συναντήσει). Εάν αποτύχει το validation, ξανακάνει την διαδικασία από την αρχή (όπως και στις optimistic και lazy υλοποιήσεις).Η υλοποίηση αυτή, χάρη στην πλήρη αποφυγή χρήσης κλειδωμάτων, φέρνει καλύτερα αποτελέσματα, αφού πλέον ούτε υπάρχει το overhead των locks, ούτε η όποια "περιττή" αναμονή άλλου νήματος που μπορεί να είχαμε στις προηγούμενες υλοποιήσεις (ιδίως στις πρώτες τρεις).

Ενότητα 3: Παραλληλοποίηση και βελτιστοποίηση αλγορίθμων σε επεξεργαστές γραφικών

Παραλληλοποίηση και βελτιστοποίηση του αλγορίθμου K-means

Ζητούμενο στην συγκεκριμένη άσκηση είναι η εξοικοίωση της γλώσσας CUDA για προγραμματιμσό σε επεξεργσατές κάρτες γραφικών (GPU) ώστε να επιταχύνουμε τον αλγόριθμο K-means.Το πρώτο βήμα είναι να τρεξουμε τον γραμμικό αλγόριθμο kmeans που μας δίνεται και παρατηρούμε ότι για να ολοκληρωθεί χρειάζεται 9812 ms για 16 clusters και 2 διαστάσεις.Επιπλέον τα κέντρα που έχουν τελικά τα clusters μετά την γραμμική υλοποιήση του kmeans θα τα χρησιμοποιήσουμε ως golden values ώστε να διαπιστώσουμε στην συνέχεια ότι οι υλοποιήσεις μας λειτουργούν σωστά.

Naive version

Η λογική πίσω απο την χρήση GPU ως επιταχυντή είναι ότι θεωρούμε οτι η CPU μας είναι ο host ο οποίος δέχται τα δεδομένα απο τον χρήση και στέλνει όσα χρειάζονται επεξεργασία σε ενα device (GPU) το οποίο τα επεξεργάζεται και τα επιστρέφει .Συγκεκριμένα η cpu στέλνει τα δεδομένα στην gpu κα τα διάφορα cores της κάνουν τους απαραίτητους υπολογισμούς παράλληλα.

```
    const unsigned int numThreadsPerClusterBlock = (numObjs > blockSize)? blockSize:
    numObjs;
    const unsigned int numClusterBlocks = (numObjs + numThreadsPerClusterBlock - 1) /
    numThreadsPerClusterBlock; //Grid Size= Threads per Block/Total Number of Threads -
    new
    const unsigned int clusterBlockSharedDataSize = numThreadsPerClusterBlock*sizeof(int)
    ; //actually the only shared variable we want all blocks see is delta so we put only
    that in mainmemory  one for every thread
  }
```

Listing 13: Calculate Grid size

Αρχικά αφοού οριστούν οι δείκτες που θα αποθηκεύσουμε τα δεδομένα που θα σταλθούν στο device(GPU) είναι απαραίτητο να δώσουμε ως ορίσματα στην συνάρτηση που θα τρέξει στην GPU το πλήθος των threads σε ένα block , το πλήθος των block που θα δεσμεύσει αυτή η συνάρτηση στην gpu καθώς και το μέγεθος της shared μνήμη,δηλαδή την κοινή μνήμη σε κάθε block που έχουν πρόσβαση όλα threads του συγκεκριμένου block .Το πλήθος των block(numClusterBlocks) υπολογίζεται με το πλήθος των αντικειμένων που θέλουμε να επεξεργαστούμε προς το πλήθος των νημάτων σε ένα block , με μια στρογγυλοποίηση προς τα πάνω ώστε σίγουρα κάθε “αντικείμενο” που θέλουμε να επεξεργαστούμε να έχει ένα thread ελεύθερο με σκοπό να παραλληλοποιηθεί η διαδικασία.Επίσης σε αυτήν την υλοποίηση το μόνο που χρειάζεται να υπάρχει σε κοινή μνήμη του κάθε block είναι το delta και καθώς το χρειαζόμαστε για κάθε νήμα μεσα στο block το μέγεθος της κοινής μνήμης είναι αυτό που φαίνεται στον κώδικα.

```
  // return success or error so we have the function checkCuda
   checkCuda(cudaMalloc(&deviceObjects, numObjs*numCoords*sizeof(double)));
   checkCuda(cudaMalloc(&deviceClusters, numClusters*numCoords*sizeof(double)));
   checkCuda(cudaMalloc(&deviceMembership, numObjs*sizeof(int)));
   checkCuda(cudaMalloc(&dev_delta_ptr, sizeof(double)));
```

Listing 14: malloc in GPU device

Στη συνέχεια, δεσμεύουμε κατάλληλη μνήμη με τη χρήση της 'cudaMalloc' για κάθε πίνακα που επιθυμούμε να μεταφέρουμε στη GPU, προκειμένου να διασφαλίσουμε τη διαθεσιμότητα χώρου. Είναι σημαντικό να σημειωθεί ότι η 'cudaMalloc' δεσμεύει χώρο στην καθολική μνήμη (global memory) της GPU. Δεδομένου ότι η καθολική μνήμη αποθηκεύεται σε εξωτερική μνήμη DRAM(off chip memory), παρουσιάζει πολύ αργά χαρακτηριστικά μνήμης. Ωστόσο, δεν είναι δυνατό να παρακάμψουμε τη χρήση της, καθώς είναι απαραίτητη για τη μεταφορά δεδομένων από τον υπολογιστή-ξενιστή (host) στη συσκευή (device). Σκοπός μας, λοιπόν, είναι να μειώσουμε την κίνηση δεδομένων μέσω της καθολικής μνήμης, καθώς αυτή είναι πολύ αργή. Παρόλο που το πλεονέκτημα της καθολικής μνήμης είναι η μεγάλη της χωρητικότητα σε σύγκριση με την μνήμη DRAM σε έναν απλό υπολογιστή.

```
    do {
        timing_internal = wtime();
        timing_cpu_gpu_transfers = wtime();
        checkCuda(cudaMemcpy(deviceClusters, clusters, numClusters*numCoords*sizeof(
    double), cudaMemcpyHostToDevice));
        checkCuda(cudaMemset(dev_delta_ptr, 0, sizeof(double)));
        timing_cpu_gpu_transfers  =  wtime() - timing_cpu_gpu_transfers;
        t1 +=timing_cpu_gpu_transfers;

       timing_gpu_part = wtime();

       find_nearest_cluster // function that runs in kernel device
       <<< numClusterBlocks, numThreadsPerClusterBlock, clusterBlockSharedDataSize >>>
            //<<< numBlocks, threadsPerBlock, sharedMemorySize >>>
        (numCoords, numObjs, numClusters,deviceObjects, deviceClusters, deviceMembership,
     dev_delta_ptr);
          // arguments of function

        cudaDeviceSynchronize();  //waits until all streams on the device have completed
    their execution , ensure proper synchronization between the CPU and GPU.
        checkLastCudaError(); //etrieves the last error that occurred on the GPU (custom
    function )

        timing_gpu_part =wtime() - timing_gpu_part ;/
        t2 += timing_gpu_part;

        timing_gpu_cpu_transfers = wtime();
        checkCuda(cudaMemcpy(membership,  deviceMembership ,numObjs*sizeof(int),
    cudaMemcpyDeviceToHost)); //new
        checkCuda(cudaMemcpy(&delta,  dev_delta_ptr  ,sizeof(double),
    cudaMemcpyDeviceToHost)); //new !!the delta is not pointer so want by value
        timing_gpu_cpu_transfers  = wtime() - timing_gpu_cpu_transfers;
        t3 +=timing_gpu_cpu_transfers;

       /* CPU part: Update cluster centers*/
       // ......//
      delta /= numObjs;
      loop++;
      timing_cpu_part =wtime() - timing_cpu_part ;//new
      t4 += timing_cpu_part;
      timing_internal = wtime() - timing_internal;
} while (delta > threshold && loop < loop_threshold);
```

Listing 15: CPU and GPU communucation

Με τη χρήση της συνάρτησης 'cudaMemcpy()', μεταφέρουμε τα δεδομένα από τη CPU (host) στη GPU (device). Αρχικά, αντιγράφουμε όλα τα δεδομένα που υπάρχουν στον πίνακα 'clusters[numClusters][numCoords]' στην καθολική μνήμη της GPU (global memory).
Στη συνέχεια, καλούμε τη συνάρτηση 'find-nearest-cluster', η οποία εκτελείται στη GPU. Για να την χρησιμοποοιήσουμε χρειάζεται να παράσχουμε τις αρχικές συνθήκες , ορίζουμε τον αριθμό των blocks και τον αριθμό των threads που θα χρησιμοποιήσει κάθε block. Επιπλέον, ορίζουμε το μέγεθος της κοινής μνήμης (shared memory). Τα δεδομένα που βρίσκονται στην κοινή μνήμη είναι διαθέσιμα και κοινά για όλα τα νήματα (threads) του block που ανήκουν σε αυτό.
Η συνάρτηση 'find-nearest-cluster' τρέχει για όλα τα threads παράλληλα. Όταν ολοκληρώσουν την αναζήτηση του πλησιέστερου cluster για όλα τα αντικείμενα, χρησιμοποιούμε την εντολή 'cudaDeviceSynchronize()' για να διασφαλίσουμε ότι όλα τα νήματα έχουν ολοκληρώσει την εκτέλεσή τους.
Αφού τα δεδομένα είναι έτοιμα, με την εντολή 'cudaMemcpy()' και το flag 'cudaMemcpyDeviceToHost', αντιγράφουμε τα αποτελέσματα (τον πίνακα 'Membership' και τη μεταβλητή 'delta') από τη GPU (device) στη CPU (host). Έπειτα, η CPU ενημερώνει τους πίνακες 'clusters', και η διαδικασία ανταλλαγής δεδομένων και υπολογισμών στη GPU επαναλαμβάνεται μέχρι να συγκλίνει η τιμή της μεταβλητής 'delta' ή να ξεπεραστεί ο προκαθορισμένος αριθμός επαναλήψεων.
Αφού λοιπόν ξεκαθαρίσαμε το πως μεταφέρουμε δεδομένα μεταξύ CPU και GPU και πως μπορούμε να αξιοποιήσουμε την GPU για να επιταχύνουμε τον αλγόριθμο ,απομένει να δούμε την προγραμματιστική λογική ώστε η συνάρτηση find-nearest-clusters να υλοποιηθεί σωστά και να τρέξει στην GPU.

```
_global__ static
void find_nearest_cluster(int numCoords,
                          int numObjs,
                          int numClusters,
                          double *objects,           //  [numObjs][numCoords]
                          double *deviceClusters,    //  [numClusters][numCoords]
                          int *deviceMembership,          //  [numObjs]
                          double *devdelta)
{

        /* Get the global ID of the thread. */
    int tid = get_tid();

        /* TODO: Maybe something is missing here... should all threads run this? */
    if (tid < numObjs) { // max tid we can have
        int   index, i;
        double dist, min_dist;
        int objectId , clusterId ;
        int local_tid = threadIdx.x; // eg local_tid take values :0,1,2,3,0,1,2,3,...
    global_tid take values:0,1,2,3,4,5,6,7,...
        extern __shared__ int devDelta_arr[]; // extern because we dont know how many
    elemetns has

        /* find the cluster id that has min distance to object */
        index = 0;
        /* TODO: call min_dist = euclid_dist_2(...) with correct objectId/clusterId */
        objectId  = tid; // every CUDA core have one tid  (global_tid)
        clusterId = 0; // just for the first cluster to initialize min_dist !
        min_dist = euclid_dist_2(numCoords, numObjs,numClusters,objects,deviceClusters,
    objectId,clusterId);

        for (i=1; i<numClusters; i++) {
            /* TODO: call dist = euclid_dist_2(...) with correct objectId/clusterId */
```

```
            objectId  = tid; // we are in the same CUDA core so the same object --this
    line it doesnt need it
            clusterId = i;  // we will comptude for this one object(tid) the distance
    from all  clusters
            dist = euclid_dist_2(numCoords, numObjs,numClusters,objects,deviceClusters,
    objectId,clusterId );
          /* no need square root */
          if (dist < min_dist) { /* find the min and its array index */
              min_dist = dist;
              index    = i;
          }
        }

         devDelta_arr[local_tid]=0; //here why this?(think again)
        if (deviceMembership[tid] != index) {
   // here we want atomic , but we choose better solution with shared memory array
             devDelta_arr[local_tid] = 1;
        }

        /* assign the deviceMembership to object objectId */
        deviceMembership[tid] = index;

        __syncthreads();
         i = blockDim.x/2;
        while(i!=0) {
            if(local_tid < i)
               devDelta_arr[local_tid] +=  devDelta_arr[local_tid+i];
            __syncthreads();
            i /=2;
        }
        if (local_tid == 0) {
            atomicAdd(devdelta, devDelta_arr[0]);
        }
    }
}
```

Listing 16: GPU function find_nearest_cluster

Παρατηρούμε ότι η συνάρτηση δέχεται ως εισόδους τους πίνακες 'objects' και 'deviceClusters' και υπολογίζει τους πίνακες 'deviceMembership' και 'devicedelta'. Συνεπώς, η λογική είναι κάθε thread να αναλαμβάνει ένα object και να βρίσκει για αυτό το κοντινότερο cluster παραλληλοποιώντας τον υπολογισμό. Στην GPU έχουμε blocks και κάθε block διαθέτει έναν συγκεκριμένο αριθμό από threads. Κάθε thread έχει ένα μοναδικό id, το global id, το οποίο υπολογίζεται ως εξής:

$$\text{global id} = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}$$

Έπειτα, αφού βρούμε το global id του thread, χρειάζεται μια συνθήκη ελέγχου ώστε να εξασφαλίσουμε ότι το τρέχων thread δεν είναι μεγαλύτερο του object, δηλαδή ότι υπάρχει διαθέσιμο object για να γίνει ο υπολογισμός του nearest cluster. Στη συνέχεια, θα βρούμε την απόσταση του object με αριθμό global id από όλα τα clusters που υπάρχουν και θα κρατήσουμε την ελάχιστη απόσταση και τη θέση που την βρήκαμε. Θα αποθηκεύσουμε τη θέση που βρήκαμε, δηλαδή σε ποιο cluster ανήκει το object με τιμή global id στον πίνακα 'deviceMembership', ο οποίος στη συνέχεια αντιγράφεται από την GPU στην CPU. Για τον υπολογισμό της απόστασης από ένα object σε ένα συγκεκριμένο cluster χρησιμοποιούμε τη συνάρτηση 'euclid-dist-2', η οποία υπολογίζει το τετράγωνο της ευκλείδιας απόστασης, μέσω ενός βρόχου για όλες τις διαστάσεις του cluster. Αξίζει να σημειωθεί ότι οι πίνακες

‘objects‘ και ‘clusters‘ που περιέχουν τις συντεταγμένες του σημείου που εξετάζουμε και όλων των κέντρων των cluster, παρά το γεγονός ότι είναι μονοδιάστατοι, τους αντιμετωπίζουμε ως δύο διαστάσεων (objects[numObjs][numCoords], clusters[numClusters][numCoords]) με πρώτη διάσταση το πλήθος και δεύτερη τις διαστάσεις.
Συνεπώς, μέχρι τώρα έχουμε σχολιάσει το πως η GPU αναθέτει σε κάθε thread ένα υπολογισμό και στο τέλος ενημερώνει διαφορετική θέση του πίνακα ‘deviceMembership‘ που βρίσκεται στην global memory όπου έχουν πρόσβαση όλα τα blocks και συνεπώς όλα τα νήματα.
Στη συνέχεια, προχωράμε στον υπολογισμό και τη διαχείριση του δείκτη delta, ο οποίος αποθηκεύεται στη shared memory. Κάθε block έχει τη δική του μνήμη που χωράει ένα delta για κάθε thread στο block. Το στοιχείο αυτό είναι προσβάσιμο σε όλα τα threads του block. Για να επιτευχθεί αυτό, πρέπει να γνωρίζουμε το local id του thread εντός του συγκεκριμένου block, το οποίο είναι το threadid.x. Επιπλέον, ορίζουμε έναν πίνακα shared για να αποθηκεύσουμε τα δεδομένα στη shared memory. Με τη βελτιωμένη λύση reduction του delta σε CUDA, έχουμε καλύτερη απόδοση από την απλή χρήση ατομικών εντολών. Είναι σημαντικό να σημειωθεί ότι η shared memory είναι γρηγορότερη από την global, αλλά είναι σημαντικά αρκετά μικρότερη και πιο δύσκολα την διαχειρήσιμη απο προγραμματιστική άποψη ώστε να εισάγουμε και τον πίνακα deviceMembership.
Τέλος απομένει να εκτελέσουμε την υλοποιήση μας,όντως παρατηρούμε ότι οι τελικοί υπολογισμοί συμφωνούν με την γραμμική υλοποιήση και έχουμε βελτίσωη του speedup όμως απο τα 64 νήματα και πάνω το speedup και ο χρόνος εκτέλεσης δεν φαίνεται να έχει σημαντικές βελτιώσεις παραμένει πρακτικά ο ίδιος που σημαίνει ότι αξιποιούμε ελάχιστα thread της GPU και δεν μπρούμε να την εκμεταλετούμε πλήρως.Φαίνεται ότι ο συγκεκριμένος αλγόριθμος δεν μπορεί να αξιοποιήσει παραπάνω πόρους ωστε να έχει βελτίωση (οπως φαίνεται στο figure 10).

```
__device__ int get_tid(){
        /* TODO: Calculate 1-Dim global ID of a thread */
      int globalId = threadIdx.x + blockIdx.x * blockDim.x;
      return globalId;
}
/* square of Euclid distance between two multi-dimensional points */
__host__ __device__ inline static
double euclid_dist_2(int    numCoords,
                     int    numObjs,
                     int    numClusters,
                     double *objects,     // [numObjs][numCoords]
                     double *clusters,    // [numClusters][numCoords]
                     int    objectId,
                     int    clusterId)
{
    int i;
    double ans=0.0;
        /* TODO: Calculate the euclid_dist of elem=objectId of objects from elem=
    clusterId from clusters*/
        for(i=0; i<numCoords; i++){
                        // in the serial we have a for loop for all dimensions ,now we
    just just take the distance between object and cluster
          ans += (objects[objectId*numCoords+i]-clusters[clusterId*numCoords+i])*(
    objects[objectId*numCoords+i]-clusters[clusterId*numCoords+i]);
        }
    return(ans);
}
```

Listing 17: device functions

transpose version

Σε αυτό το σημείο θα βελτιώσουμε την παραπάνω υλοποιήση αναλάσοντας τους την δεικτοδότηση των πινάκων,αυτό δεν θα έχει κάποια αλλαγή στα δεδομένα μας αλλα θα επιρεάσει τον τρόπο προσπέλαση της απο την global memory στην GPU.
Στην συνέχεια θα σχολιάσουμε τις αλλαγές και τις πρσοθήκες του προηγούμενου κώδικα.
Αρχικά χρησιμοποιώντας την συνάρτηση calloc2d θα ορίσουμε τρείς δισδιάστατους πίνακες με την πρώτη διάσταση των πινάκων να ορίζει τις διαστάσεις του προβλήματος και την δεύτερη το πλήθος των δεδομένων που αντιστοιχούν στον κάθε πίνακα. Το αμέσως επόμενο βήμα είναι να υπολογίσουμε τον "ανάστροφο" πίνακα του objects[i][j] και να τον τοποθετήσουμε στον νέο δισδιάστατό πίνακα που αρχικοποιήσαμε προγουμένως στο dimObjects[j][i] ,προσέχοντας το γεγονός ότι οι πίνακες που έχουμε ως εισόδους στην CPU είναι μονοδιάστατοι.

```
   /* TODO: Transpose dims */
    double  **dimObjects =  (double**)calloc_2d(numCoords,numObjs,sizeof(double));
    //calloc_2d(...) -> [numCoords][numObjs]
    double  **dimClusters = (double**)calloc_2d(numCoords,numClusters,sizeof(double));
    //calloc_2d(...) -> [numCoords][numClusters]
    double  **newClusters = (double**)calloc_2d(numCoords,numClusters,sizeof(double));
    //calloc_2d(...) -> [numCoords][numClusters]

    double *deviceObjects;
    double *deviceClusters;
    int *deviceMembership;

    printf("\n|-----------Transpose GPU Kmeans------------|\n\n");

    //  TODO: Copy objects given in [numObjs][numCoords] layout to new [numCoords][
    numObjs] layout
    // input  objects [numObjs][numCoords]  --transpose-->  ouput dimObjects  [numCoords
    ][numObjs]
        for(int i =0 ;i< numObjs;i++ )
           for (int j =0;j<numCoords;j++){
               dimObjects[j][i] = objects[i*numCoords+j]; //objetcs[i][j]
```

Listing 18: Transpose 1D to 2D

Τα δεδομένα αυτού του νέου transpose πίνακα χρειάζεται λοιπόν να στείλουμε πλέον στην GPU,για αυτό και αλλάζουμε τις εντολές cudaMemcpy.Συγκεκριμένα θα στείλουμε ακριβώς το ίδιο πλήθος δεδομένων με τον ίδιο τύπο αλλά θα τα στείλουμε με διαφορετική σειρά , οπότε αλλάζει μόνο ο δείκτης απο τον οποίο στέλνουμε τα δεδομένα και επειδή τώρα οι πίνακες είναι δύο διαστάσεων θα αρχίσουμε να αντιγράφουμε τα δεδομένα απο πρώτη γραμμή του πίνακα δηλαδή περνάει στην global memory συντεταγμένες για κάθε object μετά για το επόμενη συντεταγλενη μέχρι να φτάσσουμε στη τελευταία.

```
do {
      timing_internal = wtime();


         timing_cpu_gpu_transfers = wtime();

        checkCuda(cudaMemcpy(deviceClusters, dimClusters[0], numClusters*numCoords*sizeof
    (double), cudaMemcpyHostToDevice)); //new
        checkCuda(cudaMemset(dev_delta_ptr, 0, sizeof(double)));
         timing_cpu_gpu_transfers  =  wtime() - timing_cpu_gpu_transfers;
         t1 +=timing_cpu_gpu_transfers;
```

```
    timing_gpu_part = wtime();
      find_nearest_cluster
          <<< numClusterBlocks, numThreadsPerClusterBlock, clusterBlockSharedDataSize
   >>>
          (numCoords, numObjs, numClusters, deviceObjects, deviceClusters,
   deviceMembership, dev_delta_ptr);
       cudaDeviceSynchronize(); checkLastCudaError();
    timing_gpu_part =wtime() - timing_gpu_part ;//new
    t2 += timing_gpu_part;

        timing_gpu_cpu_transfers = wtime();
        checkCuda(cudaMemcpy(membership,  deviceMembership ,numObjs*sizeof(int),
   cudaMemcpyDeviceToHost)); //new


   checkCuda(cudaMemcpy(&delta,  dev_delta_ptr  ,sizeof(double), cudaMemcpyDeviceToHost)
   );//new !!the delta is not pointer so want by value
   timing_gpu_cpu_transfers  = wtime() - timing_gpu_cpu_transfers;
   t3 +=timing_gpu_cpu_transfers;

   /* CPU part: Update cluster centers*/
       /*......
       same as naive but using 2D transpose arrays
       .......*/
 } while (delta > threshold && loop < loop_threshold);
```

Listing 19: Transpose while loop

Μετά το τέλος του while loop έχουν υπολογιστεί και αποθηκευτεί όλα τα νέα κέντρα στο πίνακα clusters επειδή όμως η συνάρτηση kmeans της CPU επιστρέφει έναν μονοδιάστατο πίνακα με πρώτη διάσταση το πλήθος και δεύτερη την συντεταγμένη υπολογίζουμε όπως και πριν τον ανάστροφο πίνακα.

```
  /*TODO: Update clusters using dimClusters. Be carefull of layout!!! clusters[
    numClusters][numCoords] vs dimClusters[numCoords][numClusters] */
    // input dimClusters[numCoords][numClusters]  ---transpose--->  ouput clutsers[
    numClusters][cumCoords]
    for( i =0 ;i<numClusters;i++ )
           for ( j =0;j<numCoords;j++)
               clusters[i*numCoords+j] =  dimClusters[j][i]; //!beware clusters is 1D
    clusters[i][j] =  dimClusters[j][i]
```

Listing 20: Transpose 2D to 1D

```
__host__ __device__ inline static
 double euclid_dist_2_transpose(int numCoords,
                    int    numObjs,
                    int    numClusters,
                    double *objects,     // [numCoords][numObjs]
                    double *clusters,    // [numCoords][numClusters]
                    int    objectId,
                    int    clusterId)
{
    int i;
    double ans=0.0;

        /* TODO: Calculate the euclid_dist of elem=objectId of objects from elem=
    clusterId from clusters, but for column-base format!!! */
```

```
     for(i=0; i<numCoords; i++){
             // in the serial we have a for loop for all dimensions ,now we just just
    take the distance between object and cluster
            ans += (objects[objectId+i*numObjs]-clusters[clusterId+i*numClusters])*(
    objects[objectId+i*numObjs]-clusters[clusterId+i*numClusters]);
           // ans += (objects[i][objectId]-clusters[i][clusterId])*(objects[i][objectId]-
    clusters[i][clusterId]);
        }

    return(ans);
}
```

Listing 21: euclid_dist_2_transpose

Η μετατόπιση (transpose) των δεδομένων στη μνήμη της GPU οδηγεί σε μείωση του χρόνου εκτέλεσης σε σύγκριση με την απλή προσέγγιση, όπως αναμέναμε. Αυτό συμβαίνει επειδή η διάταξη των δεδομένων στη global memory είναι έτσι ώστε να λαμβάνονται πρώτα όλες οι τιμές μιας συντεταγμένης (x) για κάθε αντικείμενο, και στη συνέχεια όλες οι τιμές της επόμενης συντεταγμένης (y) για κάθε αντικείμενο, και ούτω καθεξής.
Αυτή η διάταξη επιταχύνει τη διαδικασία καθώς τα νήματα που εργάζονται παράλληλα χρησιμοποιούν παρόμοια δεδομένα και μπορούν να ανακτήσουν μια ολόκληρη γραμμή από τη μνήμη και να την καταχωρήσουν στην τοπική μνήμη κάθε νήματος. Αυτό οδηγεί σε μεγαλύτερο χρόνο ανάκτησης για μια γραμμή από τη μνήμη, αλλά λιγότερες προσβάσεις στη μνήμη οι οποίες κοστίζουν πολύ(latency),διότο η πρόσβαση στην global memory οποία είναι πολύ αργή.
Στην απλή προσέγγιση, αντίθετα, έχουμε μικρότερο χρόνο ανάκτησης αλλά περισσότερες προσβάσεις στη μνήμη για κάθε νήμα. Είναι σημαντικό να σημειωθεί ότι όλα τα νήματα εκτελούν τους υπολογισμούς τους παράλληλα, χρησιμοποιώντας τα ίδια δεδομένα την ίδια στιγμή για το αντικείμενό τους.
Ωστόσο, παρατηρούμε ότι όσο αυξάνεται το μέγεθος του block, δεν υπάρχει βελτίωση της απόδοσης και συνεχίζει να μην αξιοποιείται πλήρως η κάρτα γραφικών, όπως και στην naive προσέγγιση.

shared version

Σε αυτήν την υλοποιήση θα βελτιώσουμε την transpose υλοποιήση τοποθετώντας τα clusters στην shared memory της GPU με σκοπό να μειώσουμε τις προσβάσεις στην global μνήμη που έχουν μεγάλο latency.
Το πρώτο βήμα είναι να ορίσουμε στην συνάρτηση που τρέχει στο kernel της GPU να δεσμεύσει κατάλληλο χωρο shared memory.

```
   const unsigned int clusterBlockSharedDataSize ,DeltaSize,ClustersSize;
   DeltaSize =  numThreadsPerClusterBlock*sizeof(int);
   ClustersSize = numCoords*numClusters*sizeof(double)
   clusterBlockSharedDataSize  = DeltaSize + ClusterSize;
```

Listing 22: shared Memory size

Σημαντικό είναι να αντιγραφούν οι τιμές των clusters απο το global memory στο shared memory για κάθε block.Για να μην είναι χρονοβόρα αυτή η διαδικασία παραλληλοποιείτε στα νήματα του κάθε block με τέτοιο τρόπο ώστε κάθε νήμα του block να αντιγράφει διαφορετικά δεδομένα όπως φαίνεται στον κώδικα.Όταν ολοκληρωθεί αυτή η διαδικασία απο όλα τα νήματα το οποίο διασφαλίζουμε με την εντολή syncthreads() συνεχίσουμε τον αλγόριθμο ακριβώς όπως ήταν και προηγουμενως για τον υπολογισμό του nearest cluster.Η μόνη διαφορα είναι ότι πλέον αντι να αντλούμε τα δεδομένα

μας απο τον deviceClusters (global memory) τα αντλούμε απο τον πίνακα shemClusters (shared memory). Αξίζει να σημειώσουμε οτι χρησιμοποιούμε ξεχωριστούς shared arrays για τα δεδομενα του sharedCluster και τα δεδομένα του sharedDelta .

```
  __global__ static
void find_nearest_cluster(int numCoords,
                          int numObjs,
                          int numClusters,
                          double *objects,           //  [numCoords][numObjs]
                          double *deviceClusters,    //  [numCoords][numClusters]
                          int *deviceMembership,          //  [numObjs]
                          double *devdelta)
{
  extern __shared__ double shmemClusters[];
  double *shmemClusters_ptr =  shmemClusters;

  /* TODO: Copy deviceClusters to shmemClusters so they can be accessed faster.
    BEWARE: Make sure operations is complete before any thread continues... */

  int i = threadIdx.x;
    int b = blockDim.x;

  while(i < numCoords*numClusters) {
          shmemClusters[i] = deviceClusters[i];
                i += b;
  }

  __syncthreads();

  /* Get the global ID of the thread. */
    int tid = get_tid();

  /* TODO: Maybe something is missing here... should all threads run this? */
    if (tid < numObjs) {
        int   index, i;
        double dist, min_dist;
  int objectId , clusterId ;
        int local_tid = threadIdx.x;

  extern __shared__ int devDelta_arr[];
  int *devDelta_ptr = devDelta_arr;

        /* find the cluster id that has min distance to object */
        index = 0;
        /* TODO: call min_dist = euclid_dist_2(...) with correct objectId/clusterId using
     clusters in shmem*/

  objectId  = tid; // every CUDA core have one tid  (global_tid)
        clusterId = 0; // just for the first cluster to initialize min_dist !
  min_dist = euclid_dist_2_transpose(numCoords, numObjs,numClusters,objects,shmemClusters
    ,objectId,clusterId);

        for (i=1; i<numClusters; i++) {
            /* TODO: call dist = euclid_dist_2(...) with correct objectId/clusterId using
     clusters in shmem*/
           objectId  = tid;
              clusterId = i;  // we will comptude for this one object(tid) the distance
    from all  clusters
```

```
            dist = euclid_dist_2_transpose(numCoords, numObjs,numClusters,objects,
shmemClusters,objectId,clusterId );

        /* no need square root */
        if (dist < min_dist) { /* find the min and its array index */
            min_dist = dist;
            index    = i;
        }
    }

/*same as naive but for array devDelta_ptr
 .....
*/
```

Listing 23: shared

Αυτή η υλοποιήση είναι πράγματι η γρηγορότερη συγκριτικά με τις προηούμενες καθώς αποφεύγονται τα accesses στην κύρια μνήμη για τον υπολογισμό της απόστασης του κάθε object από όλα τα clusters, τα οποία στοιχίζουν χρόνο λόγω του latency της global memory.Βέβαια είνα σημαντικό να αναφέρουμε πως το μέγεθος της shared memory αυξήθηκε σημαντικά για να μπορούμε να αποθηκεύουμε εκεί και τα δεδομένα τών clusters.Σε αυτήν την υλοποιήση βλέπουμε οτι το speedup αυξάνει μέχρι και για block size 128 ,καθώς θέλουμε να έχουμε λίγα blocks, διαφορετικά αν έχουμε πολλά οδηγούμαστε να αντιγράψουμε πολλές φορές τον πίνακα cluster στην shared memory καθε block.Αντιθέτως όταν έχουμε πολύ μεγάλα blocks πολλά threads χρησιμοποιούν την ίδια κοινή μνήμη, με αποτέλεσμα να έχουμε στη shared memory bank conflicts (Όταν πολλά threads προσπαθούν να προσπελάσουν δεδομένα από την ίδια κοινή μνήμη ταυτόχρονα, μπορεί να προκύψουν συγκρούσεις στις τράπεζες μνήμης (memory bank conflicts)) με τα requests προς αυτήν να μην μπορούν να εξυπηρετηθούν παράλληλα, γεγονός που δυσχεραίνει την επίδοση.

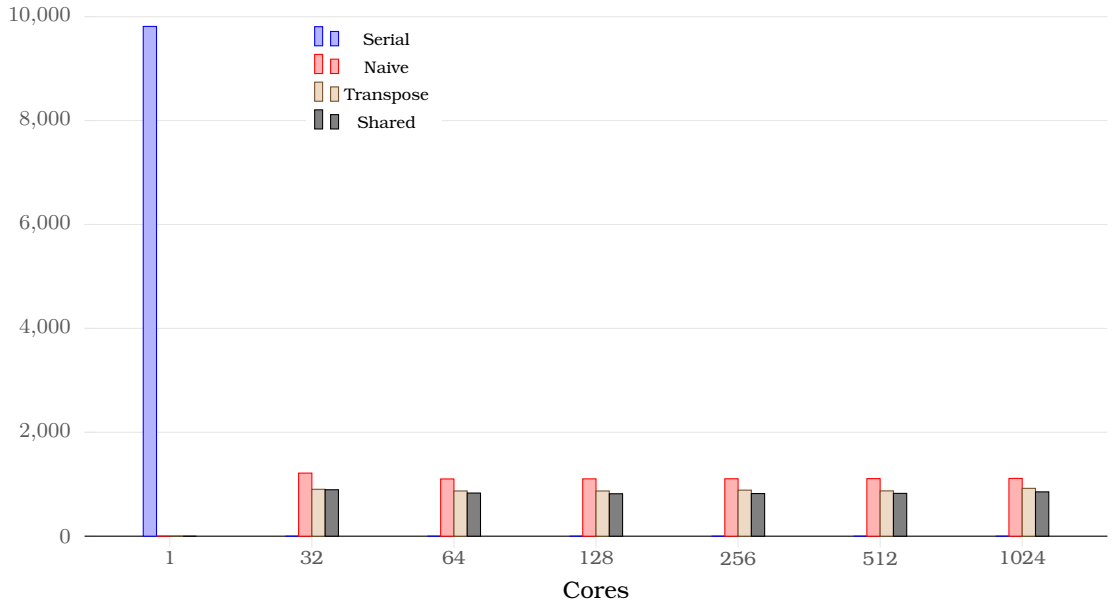


Figure 20: Performance of sequential, naive, transpose and shared versions

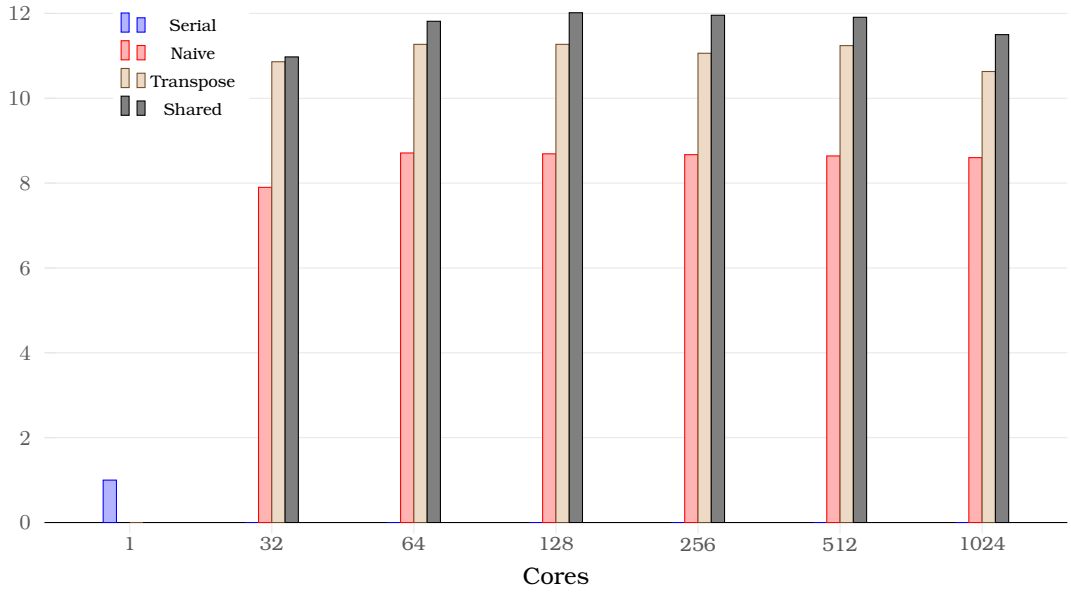
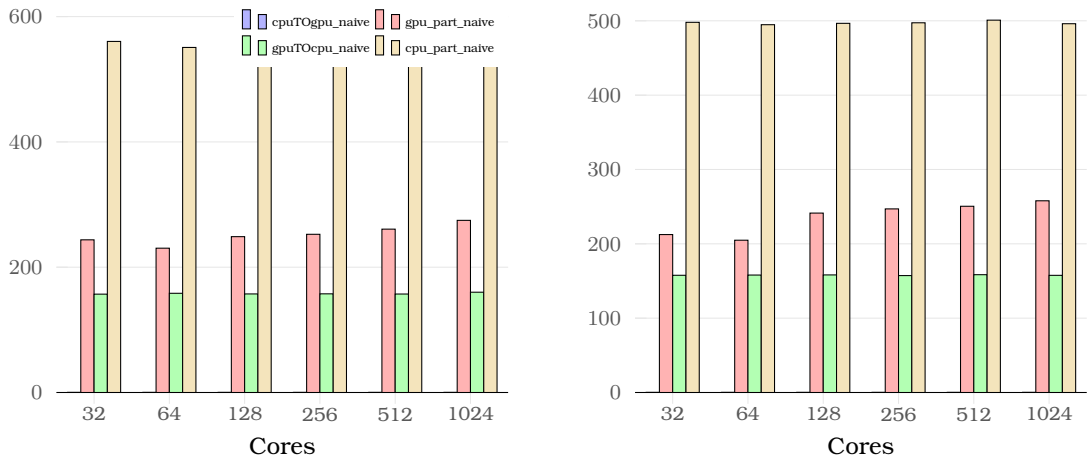


Figure 21: Speedup of sequential, naive, transpose and serial versions

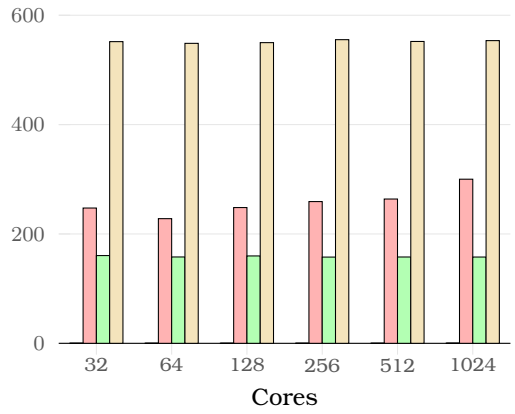
Σε γενικές γραμμές, σε όλες τις περιπτώσεις για το configuration 156 2 16 10, παρατηρούμε τα εξής:
1. Χρόνος CPU vs. GPU: Ο χρόνος εκτέλεσης στην CPU είναι σημαντικά μεγαλύτερος από τον χρόνο εκτέλεσης στην GPU. Αυτό οφείλεται στη φύση της GPU ως επιταχυντής, καθώς και στον τρόπο λειτουργίας του αλγορίθμου kmeans που επιταχύνει σημαντικά τον υπολογισμό του nearest cluster για κάθε object.
2. Χρόνος μεταφοράς δεδομένων GPU-CPU: Ο χρόνος μεταφοράς δεδομένων από τη GPU στη CPU είναι σημαντικά μεγάλος. Αυτό οφείλεται στην ανάγκη μεταφοράς δεδομένων σε κάθε επανάληψη του αλγορίθμου kmeans, κάτι που θα μπορούσε να αποφευχθεί αν μεγαλύτερο μέρος του αλγορίθμου υλοποιούνταν στη GPU.
3. Βελτιστοποιημένες υλοποιήσεις: Οι βελτιστοποιημένες υλοποιήσεις(shared,trasnposed) προκαλούν καθυστέρηση στον χρόνο εκτέλεσης της CPU, αντί για τους χρόνους εκτέλεσης των kernels ή τις μεταφορές δεδομένων. Αυτό υποδηλώνει ότι περαιτέρω βελτιστοποιήσεις πρέπει να επικεντρωθούν στην μείωση υπολογιστικού φόρτου της CPU, αντί να επικεντρωθούν στην επεξεργασία των kernels ή στις μεταφορές δεδομένων.
Η αύξηση των διαστάσεων από 2 σε 16 οδήγησε σε μείωση του συνολικού χρόνου εκτέλεσης και του speedup που επιτυγχάνεται από τη GPU. Αυτό σημαίνει ότι το μέρος των υπολογισμών που αναλαμβάνει η GPU μειώθηκε περαιτέρω σε σχέση με τις περιπτώσεις με 2 διαστάσεις.
Παρατηρήσαμε επίσης μεγαλύτερη αύξηση του speedup κατά τη μετάβαση από την naive στην transpose έκδοση, καθώς τα συγκεκριμένα coordinates των διαφορετικών αντικειμένων βρίσκονται ακόμα πιο μακριά στη μνήμη λόγω της αύξησης των διαστάσεων από 2 σε 16.
Επιπλέον, παρατηρήσαμε ότι η παρούσα shared υλοποίηση δεν είναι κατάλληλη για την επίλυση του kmeans για αυθαίρετες διαμορφώσεις, καθώς η αύξηση των διαστάσεων ή των clusters μπορεί να αυξήσει σημαντικά την κατανάλωση shared μνήμης, οδηγώντας σε πολύ χαμηλό occupancy και πιθανότατα ακόμα και την αδυναμία εκτέλεσης του kernel σε ορισμένες περιπτώσεις.

Performance for configuration 256-2-16-10



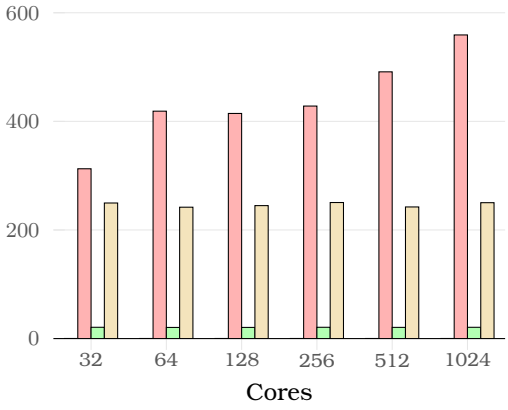
(a) Naive execution time (sec)

(b) Transpose execution time (sec)

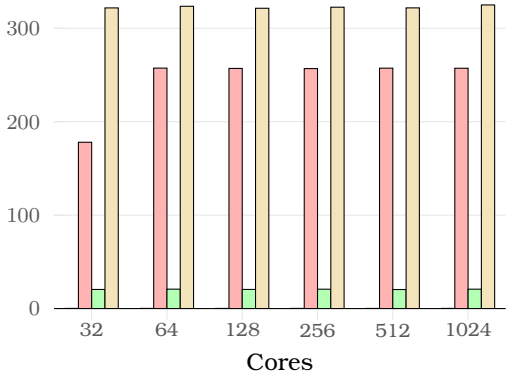


(c) Shared execution time (sec)

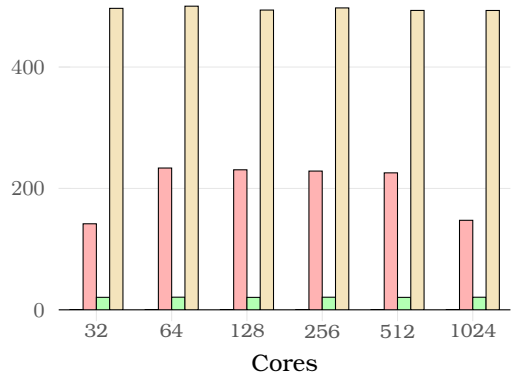
Performance for configuration 256-16-16-10



(d) Naive execution time (sec)



(e) Transpose execution time (sec)



(f) Transpose execution time (sec)

All GPU Version

Τέλος δοκιμάζουμε να "μεταφέρουμε" την υλοποιήση του υπολογισμού των νέων κέντρων που μέχρι τώρα γινόταν στην CPU σε έναν kernel στην GPU για να επιταχύνουμε την εκέλεση του αλγόριθμου. Αρχικά θα χρειαστέι να δεσμεύσουμε περισσότερο χώρο στην GPU ώστε να μπορούμε πλέον να αποθηκέουμε και τους πίνακες devicenewClusters και devicenewClusterSize που θα αξιοποιούνται για τον υπολογισμό των νέων κέντρων των clusters.Σημνατικό πλεονέκτημα είναι ότι πλεον δεν χρειάζεται να μεταφέρουμε το deviceMembership απο την cuda στην gpu καθώς ο υπολογισμός θα γίνεται μέσα στο device.Συγκεκριμένα θα δημιουργήσουμε μια ακόμη kernel function στην GPU και αυτές οι δύο θα μπορούν να έχουν η μια πρόσβαση στα δεδομένα της άλλης μέσω της global memory.Προσθέτουμε λοιπόν στην findnearestcluster δυο ακόμη ορίμστα devicenewClusterSize, devicenewClusters ώστε να ενημρώνουμε τα νέα κέντα των clusters δηλαδή για να ανικαταστήοσυμε την for loop για όλα τα objects που γινόντουσα στην CPU καθώς τώρα καθε global id (κάθε thread)επεξεργάζεται και ενα διαφορετικό object.

```
void find_nearest_cluster(int numCoords,
                          int numObjs,
                          int numClusters,
                          double *deviceobjects,           //  [numCoords][numObjs]
/*
                          TODO: If you choose to do (some of) the new centroid
    calculation here, you will need some extra parameters here (from "update_centroids").
*/                        int *devicenewClusterSize,           //  [numClusters]
                          double *devicenewClusters,    //  [numCoords][numClusters]

                          double *deviceClusters,    //  [numCoords][numClusters]
                          int *deviceMembership,          //  [numObjs]
                          double *devdelta)
{


extern __shared__ char sharedMemory[];

    // Calculate the size of each array in shared memory
    int deltaSize = blockDim.x * sizeof(int);
    int clustersSize = numClusters * numCoords * sizeof(double);
    int newClustersSize = numClusters * numCoords * sizeof(double);
    int newClustersSizeSize = numClusters * sizeof(int);

    // Pointers to access shared memory
    int *devDelta_ptr = (int *)sharedMemory;
    double *shmemClusters = (double *)(sharedMemory + deltaSize);
    double *shmemNEWClusters = (double *)(sharedMemory + deltaSize + clustersSize);
    int *shmemNEWClustersSIZE = (int *)(sharedMemory + deltaSize + clustersSize +
    newClustersSize);

     /* TODO: copy me from shared version... */
         int i = threadIdx.x;
       int b = blockDim.x;
    while(i < numCoords*numClusters) {
            shmemClusters[i] = deviceClusters[i];
            shmemNEWClusters[i] = 0; //new
            if(i<numClusters)  shmemNEWClustersSIZE[i] = 0; //new
             i += b;
   }
```

```
    __syncthreads();

    int tid = get_tid();

    if (tid < numObjs) {
        int   index, i,j;
        double dist, min_dist;
      int objectId , clusterId ;
        int local_tid = threadIdx.x;

        index = 0;
        objectId  = tid;
        clusterId = 0;
        min_dist = euclid_dist_2_transpose(numCoords, numObjs,numClusters,deviceobjects,
    shmemClusters,objectId,clusterId);

      //find the min same as shared version
      //.......

        devDelta_ptr[local_tid]=0;
        if (deviceMembership[tid] != index) {devDelta_ptr[local_tid] = 1;}

        deviceMembership[tid] = index;
     //NEW
       atomicAdd(&shmemNEWClustersSIZE[index],1);

        for(j =0;j<numCoords;j++) {
            //devicenewClusters[j][index] += deviceobjects[j][tid];
            atomicAdd(&shmemNEWClusters[j*numClusters+index],deviceobjects[j*numObjs+tid
    ]);

        }
        __syncthreads();

       //reduction for delta same as shared version
       //  ......

        //reduction of shared arrays shmemNEWClusters,shmemNEWClustersSIZE  --NEW
         i = threadIdx.x;
         b = blockDim.x;
        while(i < numCoords*numClusters) {
          atomicAdd(&devicenewClusters[i],shmemNEWClusters[i]);
            if(i<numClusters) atomicAdd(&(devicenewClusterSize[i]),shmemNEWClustersSIZE[i
    ]);
            i += b;
      }
    }
}
```

Listing 24: find_nearest_cluster_all

Όπως φαίνεται στον παραπάνω κώδικα λοιπόν τοποθετούμε όλους τους πίανκες στην shared memory (αλλάζοντας κατάλληλα το μέγεθος της προηγουμένως) ώστε να μην έχουμε καθυστερήσεις λόγο latency της επικοινωνίας με την global memory.Για να γίνει αυτό σωστά θεωρούμε ως char την shared memory (ώστε καθε αύξηση να είναι ενα byte) και τοποθετούμε ανάλογα το μέγεθος καθε πίνακα να ξεκινούν απο κατάλληλο offset ωστε να μην έχουμε επικάλυψη των πινάκων. Στην συνέχεια αρ-

χικοποιούμε τους πίνακες shmemNEWClusters και shmemNEWClustersSIZE με την ίδια λογική που αντιγράψαμε τον πίνακα shemclusters στην shared υλοποιήση απο την global στην shared memory. Αφού ολοκληρωθεί η εύρεση του κοντινότερου cluster στο object που έχει αναλάβει το thread ενημερωνει τις τιμές των πινάκων με attomic operation καθώς αυτοί βρίσκονται στην shared μνήμη και διαοφρετικά μπορεί να χαθεί κάποια ενημερωση τιμής.Τέλος απομένει να γίνει το reduction των δύο νεων πινάκων ώστε να περάσουν σωστά στην global memory για να μπορεί να τα επεξεργστεί κατάλληλα η συνάρτηση που υπολογίζει τα νεα κέντρα.

```
__global__ static
void update_centroids(int numCoords,
                          int numClusters,
                          int *devicenewClusterSize,           //  [numClusters]
                          double *devicenewClusters,    //  [numCoords][numClusters]
                          double *deviceClusters)    //  [numCoords][numClusters])
{

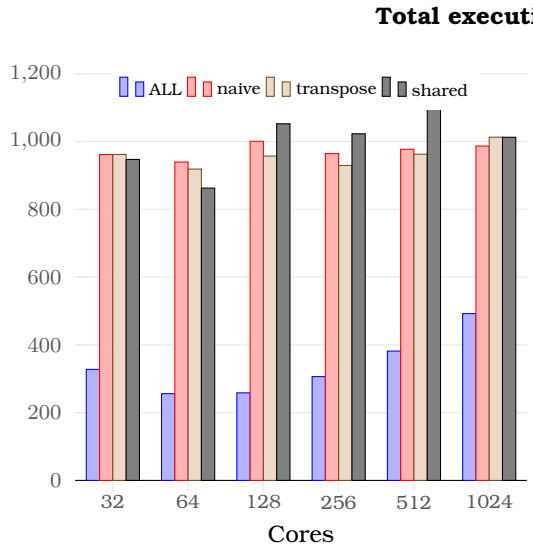
    /* TODO: additional steps for calculating new centroids in GPU? */
    int global_tid= get_tid() ;  // every thread will update one value from clustes array
    nt i,j,done,temp; // lets say i run numClusters and j numCoords

  if(global_tid <numCoords*numClusters) {

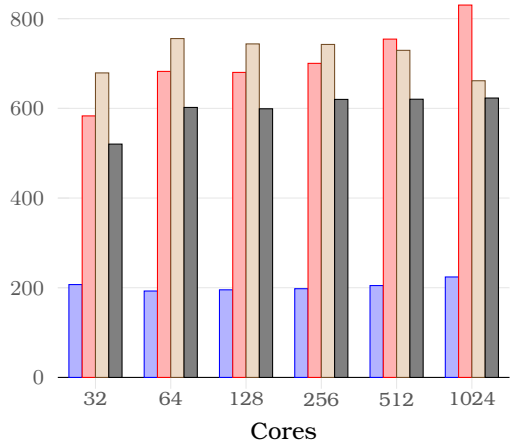
     i = global_tid%numClusters;
    if(devicenewClusterSize[i]>0) {
       // deviceClusters[j][i] = devicenewClusters[j][i]/devicenewClusterSize[i];
       deviceClusters[global_tid] = devicenewClusters[global_tid]/devicenewClusterSize[i
    ];
     }
 }
```

Listing 25: find_nearest_cluster_all

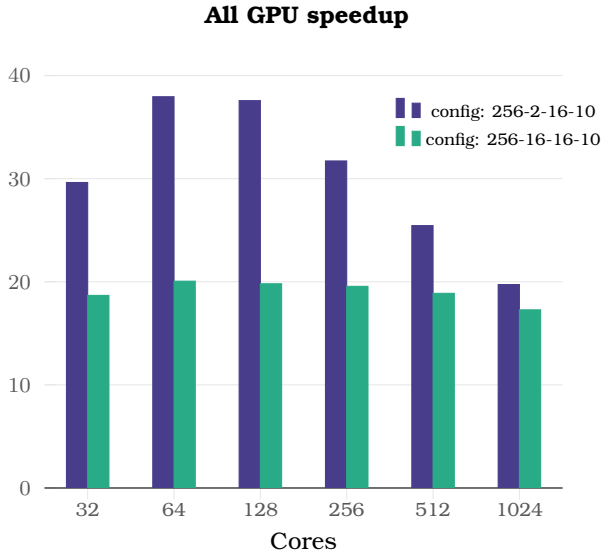
Αυτή η νεα συνάρτηση που αντικαθιστά το κομμάτι της CPU είναι υπέυθυνη για average the sum and replace old cluster centers with newClusters , την διαμορφώνουμε με την λογικη ώστε κάθε thread να υπεύθυνο να υπολογίζει ενα μόνο νεο index του πίνακα των cluster οπότε αντικαθιστά της διπλή for loop της cpu τόσο για κάθε cluster όσο και για κάθε coordinate. Το i μαδε δίνει τον αριθμό του cluster που βρισκόμαστε δηλαδή της δεύτερης διάστασης τπυ αντίστοιχου δισδιάστατου πίνακα. Αφού ολοκληρωθούν όλες οι επαναλήψεις τότε με cudaMemcpy αποστέλουμε απο την GPU τα deviceclusters στην CPU ως τελικό αποτέλεσμα.
Προφανώς τώρα δέν έχει νόημα να δούμε τους επιμέρους χρόνου καθώς σχεδόν πλέον όλες οι υλοποιήσεις πραγματοποιούνται στην CPU και η μεταφορές δεδομένων πέρα του delta έχουν εξαληφθεί μέσα στην επανάληψη. Αρχικά παρατηρούμε τεράστα βελτίωση και πολύ καλύτερους χρόνους εκτέλεσης με την υλοποιήση εξολοκλήρου του αλγόριθμου στην GPU αφου αναθέτοντας τους υπολογισμούς της CPU στην GPU εξοικονομούμε πολύ χρόνο στις πράξεις.Ενδειτικά σε κάποιες περιπτώσεις μπορεί να έχουμε 38 speedup.¨Οσον αφορά το block size παρατηρούμε ότι έχουμε μέγιστο speedup για 32 και στην συνέχεια όσο το αυξάνουμε ο χρόνος εκτέλεσης αρχίσει και αυξάνει ,ιθανότατα επειδή η υψηλή χρήση shared memory (τώρα ακόμα ψηλότερη) το περιορίζει.



(g) Performance for configuration 256-2-16-10



(h) Performance for configuration 256-16-16-10



Ενότητα 4: Παραλληλοποίηση και βελτιστοποίηση αλγορίθμων σε αρχιτεκτονικές κατανεμημένης μνήμης

Παραλληλοποίηση και βελτιστοποίηση του αλγορίθμου K-means

Στο σημείο αυτό ζητείται να παραλληλοποιήσουμε, ξανά, τον αλγόριθμο k-means, αλλά σε αρχιτεκτονική κατανεμημένης μνήμης χρησιμοποιώντας (για την επικοινωνία των διεργασιών) το MPI. Για τον σκοπό αυτόν μας δίνεται σκελετός, τον οποίο συμπληρώνουμε ως εξής:

- **Στην main:**
  Κάνουμε broadcast τις αρχικές θέσεις των cluster σε όλες τις διεργασίες (ranks)

```
/*
 * TODO: Broadcast initial cluster positions to all ranks
 */
MPI_Bcast(clusters, numClusters*numCoords, MPI_DOUBLE, 0, MPI_COMM_WORLD);
```

Listing 26: K-means broadcast in main()

Φροντίζουμε η συνάρτηση kmeans να καλείται με τα σωστά (νέα) ορίσματα:

```
// start the core computation
    /*
     * TODO: Fix number of objects that this kmeans function call will process
     */
    kmeans(objects, numCoords, rank_numObjs, numClusters, threshold, loop_threshold,
     membership, clusters);
```

Listing 27: K-means kmeans-function call in main()

Υπολογίζουμε τα recvcounts και displs, απαραίτητα για την κλήση της Gather:

```
// Gather membership information from all ranks to tot_membership
    int recvcounts[size], displs[size];
    int r;
    if (rank == 0) {
        /* TODO: Calculate recvcounts and displs, which will be used to gather data
    from each rank.
         * Hint: recvcounts: number of elements received from each rank
         *       displs: displacement of each rank's data
         */
        for (r=0; r<size; r++) {
                recvcounts[r] = (r < (numObjs % size)) ? ((numObjs/size) + 1) : (
    numObjs/size);
                displs[r] = (r == 0) ? 0 : (displs[r - 1] + recvcounts[r - 1]);
        }
    }
```

Listing 28: K-means recvcounts and displs calculation in main()

Κάνουμε broadcast τα recvcounts και displs στις υπόλοιπες διεργασίες:

```
/*
 * TODO: Broadcast the recvcounts and displs arrays to other ranks.
 */
MPI_Bcast(recvcounts, size, MPI_INT, 0, MPI_COMM_WORLD);
MPI_Bcast(displs, size, MPI_INT, 0, MPI_COMM_WORLD);
```

Listing 29: K-means recvcounts and displs broadcast in main()

Κάνουμε gather τα membership:

```
/*
 * TODO: Gather membership information from every rank. (hint: each rank may send
    different number of objects)
 */
MPI_Gatherv(membership, rank_numObjs, MPI_INT, tot_membership, recvcounts, displs,
    MPI_INT, 0, MPI_COMM_WORLD);
```

Listing 30: K-means membership gather in main()

- **Στην kmeans:**
  Εκτελούμε reduction στα clustered data (όλες οι διεργασίες το υπολογίζουν):

```
/*
 * TODO: Perform reduction of cluster data (rank_newClusters, rank_newClusterSize)
    from local arrays to shared.
 */

MPI_Allreduce(rank_newClusters, newClusters, numClusters*numCoords, MPI_DOUBLE,
    MPI_SUM, MPI_COMM_WORLD);
MPI_Allreduce(rank_newClusterSize, newClusterSize, numClusters, MPI_INT, MPI_SUM,
    MPI_COMM_WORLD);
```

Listing 31: K-means reduction on clustered data in kmeans()

Εκτελούμε reduction στις τιμές delta των διεργασιών (όλες οι διεργασίες το υπολογίζουν):

```
/*
 * TODO: Perform reduction from rank_delta variable to delta variable, that will be
    used for convergence check.
 */
MPI_Allreduce(&rank_delta, &delta, 1, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);
```

Listing 32: K-means reduction on delta in kmeans()

- **Στην file_io.c:**
  Υπολογίζουμε τον αριθμό των objects της κάθε διεργασίας:

```
/*
 * TODO: Calculate number of objects that each rank will examine (*rank_numObjs)
 */
*rank_numObjs = (rank < (numObjs % size)) ? ((numObjs/size) + 1) : (numObjs/size);
```

Listing 33: K-means rank_numObjs calculation in dataset_generation()

Υπολογίζουμε τα sendcounts και displs, απαραίτητα για την κλήση της Scatter:

```
if (rank == 0) {
    objects = (typeof(objects)) malloc(numObjs * numCoords * sizeof(*objects));

    /*
     * TODO: Calculate sendcounts and displs, which will be used to scatter data to
    each rank.
     * Hint: sendcounts: number of elements sent to each rank
     *       displs: displacement of each rank's data
     */
    int r;
    for (r=0; r<size; r++) {
            sendcounts[r] = (r < (numObjs % size)) ? (((numObjs/size) + 1) *
    numCoords) : ((numObjs/size) * numCoords);
            displs[r] = (r == 0) ? 0 : (displs[r - 1] + sendcounts[r - 1]);
    }
}
```

Listing 34: K-means sendcounts and displs calculation in dataset_generation()

Κάνουμε broadcast τα sendcounts και displs στις υπόλοιπες διεργασίες:

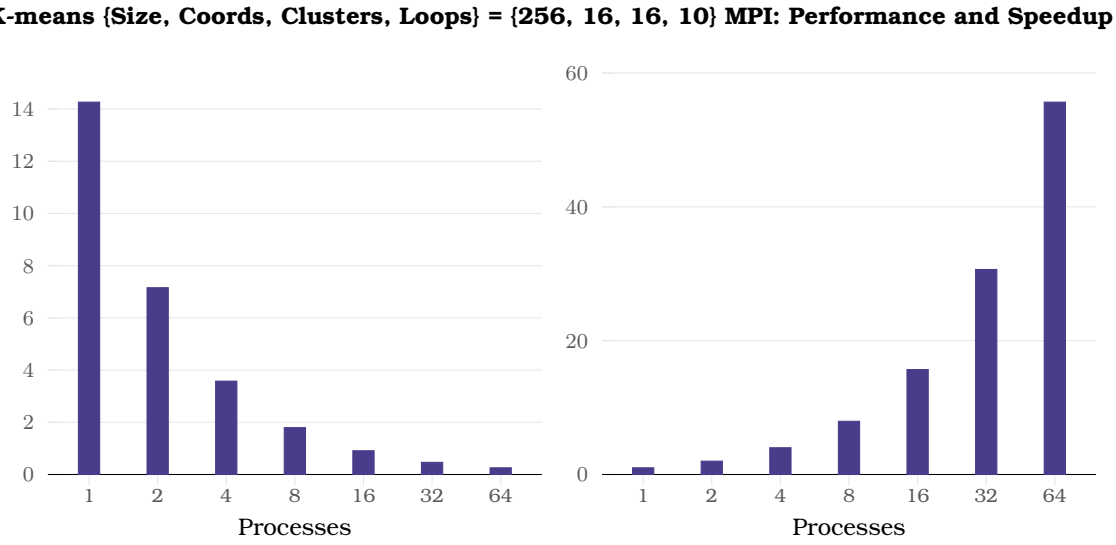
```
/*
 * TODO: Broadcast the sendcounts and displs arrays to other ranks
 */
MPI_Bcast(sendcounts, size, MPI_INT, 0, MPI_COMM_WORLD);
MPI_Bcast(displs, size, MPI_INT, 0, MPI_COMM_WORLD);
```

Listing 35: K-means sendcounts and displs broadcast in dataset_generation()

Κάνουμε scatter τα objects σε όλες τις διεργασίες

```
/*
 * TODO: Scatter objects to every rank. (hint: each rank may receive different
    number of objects)
 */
MPI_Scatterv(objects, sendcounts, displs, MPI_DOUBLE, rank_objects, rank_Objs_size,
    MPI_DOUBLE, 0, MPI_COMM_WORLD);
```

Listing 36: K-means objects scatter in dataset_generation()



(j) Performance (Execution time in sec)

(k) Speedup

Παραπάνω παρατίθενται τα γραφήματα Performance και Speedup για την εκτέλεση του k-means με configuration Size, Coords, Clusters, Loops = 256, 16, 16, 10. Όπως διαπιστώνεται, η υλοποίηση με MPI είναι scalable. Με αύξηση του αριθμού των διεργασιών, ο χρόνος σχεδόν υποδιπλασιάζεται, δηλαδή έχουμε γραμμικό speedup. Αυτό είναι αναμενόμενο, καθώς πέρα από το penalty της επικοινωνίας (που στο συγκεκριμένο παράδειγμα περιορίζεται σε τρια αρχικά broadcasts, ένα για τα initial cluster values, ένα για τα recvcounts και displs και ένα για τα sendcounts και τα αντίστοιχα displs, ένα scatter καθώς και σε ένα gather και 3 reductions ανά loop), οι υπόλοιποι υπολογισμοί είναι ανεξάρτητοι και, συνεπώς, πλήρως παραλληλοποιήσιμοι μεταξύ των διεργασιών. Έτσι, με την αύξηση των διεργασιών, το computational μέρος, που είναι σημαντικό για αυτό το πρόβλημα, εκτελείται, αναλογικά, πιο γρήγορα.

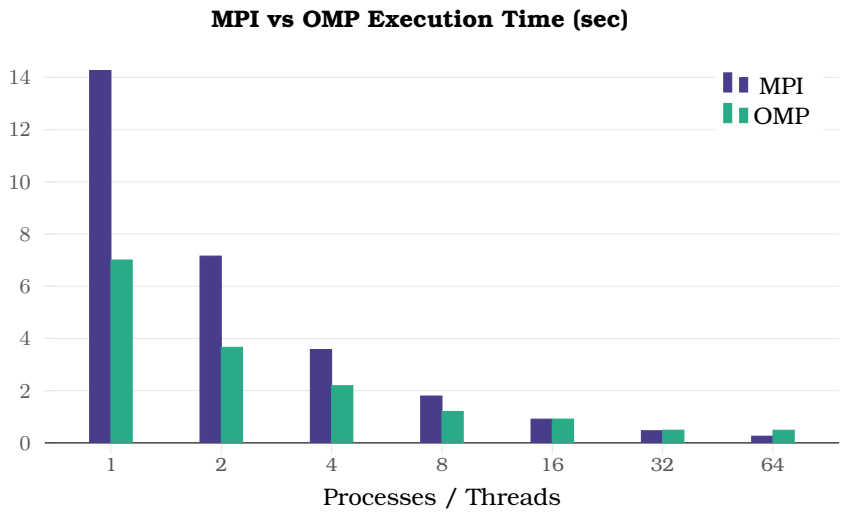


Figure 22: K-means: Comparison of MPI and OMP Performance
{Size, Coords, Clusters, Loops} = {256, 16, 16, 10}

Παραπάνω παραθέτουμε, επιπλέον, ένα διάγραμμα που περιλαμβάνει μαζί με τους χρόνους εκτέλεσης του ḱmeans με MPI (με το ίδιο configuration) και αυτούς που προέκυψαν με χρήση του OMP στην ενότητα 2 (kmeans-reduction). Όπως μπορούμε να παρατηρήσουμε, η υλοποίηση με το OMP (αρχιτεκτονική κοινής μνήμης) για μικρό αριθμό από threads είναι καλύτερη, κάτι αναμενόμενο, αφού η επικοινωνία στο ίδιο μηχάνημα είναι ταχύτερη από αυτή μέσω δικτύου διασύνδεσης. Ωστόσο, η επικοινωνία αυτή με τη αύξηση των threads δημιουργούσε συμφώρηση και, συνεπώς, επιπλέον καθυστερήσεις στο bus, ώστε το speedup να μειώνεται. Κάτι τέτοιο, ωστόσο, δεν συμβαίνει στην υλοποίηση με MPI, ώστε να καταλήγει, όπως φαίνεται στο διάγραμμα, να έχει καλύτερους χρόνους εκτέλεσης για περισσότερες από 16 διεργασίες (και προφανώς όλο και μικρότεροι χρόνοι με την αύξηση αυτών).

Παραλληλοποίηση αλγορίθμων επίλυσης του προβλήματος διάδοσης της θερμότητας σε δύο διαστάσεις

Μετρήσεις με έλεγχο σύγκλισης

Παρακάτω παραθέτουμε τα διαγράμματα τόσο του αριθμού επαναλήψεων του βασικού loop όσο και του χρόνου εκτέλεσης (computation and communication) των αλγορίθμων Jacobi και Gauss-Seidel. Όπως αναμέναμε η μέθοδος Jacobi αργεί να συγκλίνει το οποίο απεικονίζεται τόσο απο τον σημαντικά μεγαλύτερο αριθμό των επαναλήψεων που αυτός εκτελεί όσο και απο τους χρόνους. Επίσης αξίζει να σχολιαστεί ότι και στους δύο αλγορίθμους ο χρόνος επικοινωνίας είναι πολύ σημαντικότερος απο αυτόν του υπολογισμού.Έτσι μεταξύ αυτών των δυο ο Gauss-Seidel αποτελεί την καλύτερη επιλογή (για υλοποιήση με έλεγχο σύγκλισης).

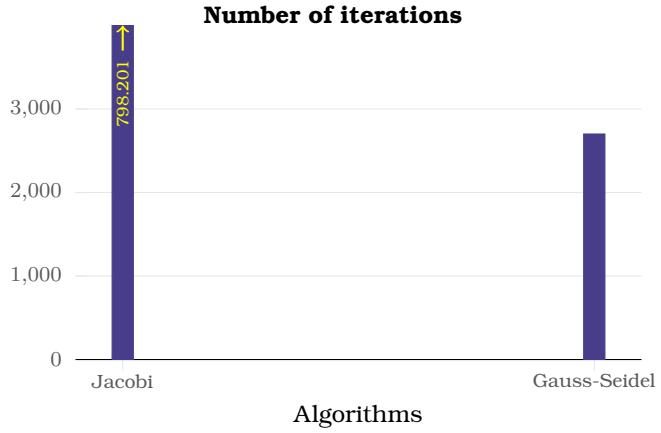


Figure 23: Heat transfer: Number of computing iterations until convergence for each algorithm

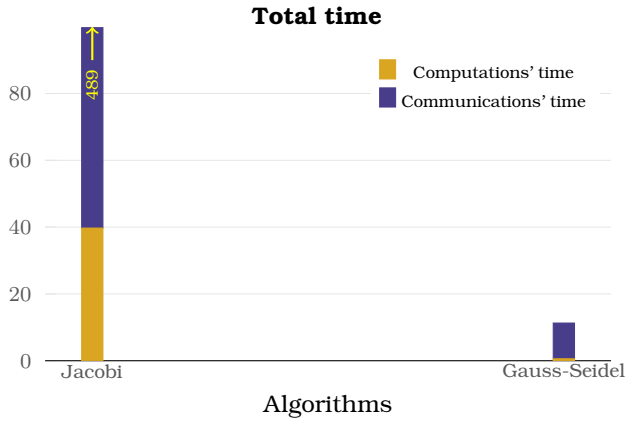


Figure 24: Heat transfer: Computations' and communications' execution time until convergence for each algorithm

Μετρήσεις χωρίς έλεγχο σύγκλισης

Στο σημείο αυτό θα συγκρίνουμε τους αλγορίθμους Jacobi και Gauss-Seidel σε υλοποιήση χωρίς έλεχγο σύγκλισης. Αρχικά, για μέγεθος προβλήματος 2048x2048, όπως φαίνεται απο το αντίστοιχο διάγραμμα του speedup και οι δύο αλγόριθμοι κλιμακώνουν, ωστόσο ο Jacobi σε σημαντικα μεγαλύτερο βαθμό.
Αυτή η συμπεριφορά είναι αναμενόμενη. Στην Jacobi, σε κάθε iteration οι διεργασίες ανταλλάσουν πρώτα τα απαραίτητα δεδομένα μεταξύ τους (στους γείτονες) και στη συνέχεια εκτελούν τους απαρταίτητους υπολογισμούς, και προφανώς το κόστος της επικοινωνίας είναι πολύ μικρότερο από το όφελος του παραλληλισμού, ώστε να έχουμε την κλιμακωσιμότητα που διακρίνεται στο διάγραμμα. Αντιθέτως, στην Gauss-Seidel κάθε process περνάει από μια ανάλογη "φάση" επικοινωνίας με τους "κάτω" και "δεξιά" γείτονες για την λήψη των "παλιών" δεδομένων, αλλά, επιπλέον, περιμένει να ολοκληρωθούν οι υπολογισμοί των "πάνω" και "αριστερά" processes και να σταλούν και οι απαραίτητες νέες (συνοριακές με αυτόν) τιμές. Έτσι, είναι αναμενόμενο ότι το speedup θα είναι σημαντικά μικρότερο, συγκριτικά με την Jacobi, ενώ οι computational χρόνοι των δύο είναι σαφώς παρόμοιοι.

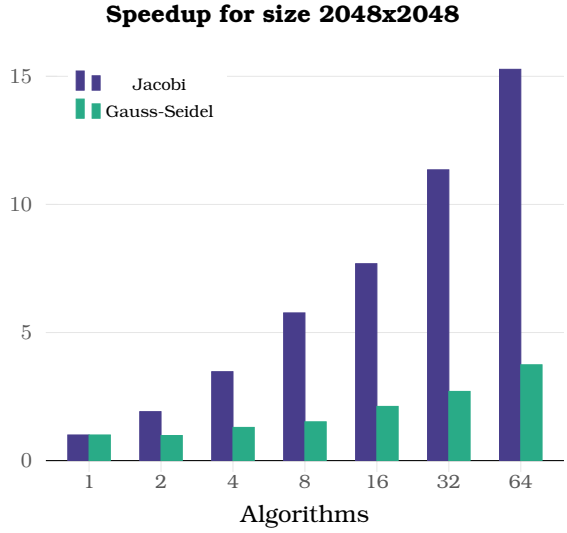
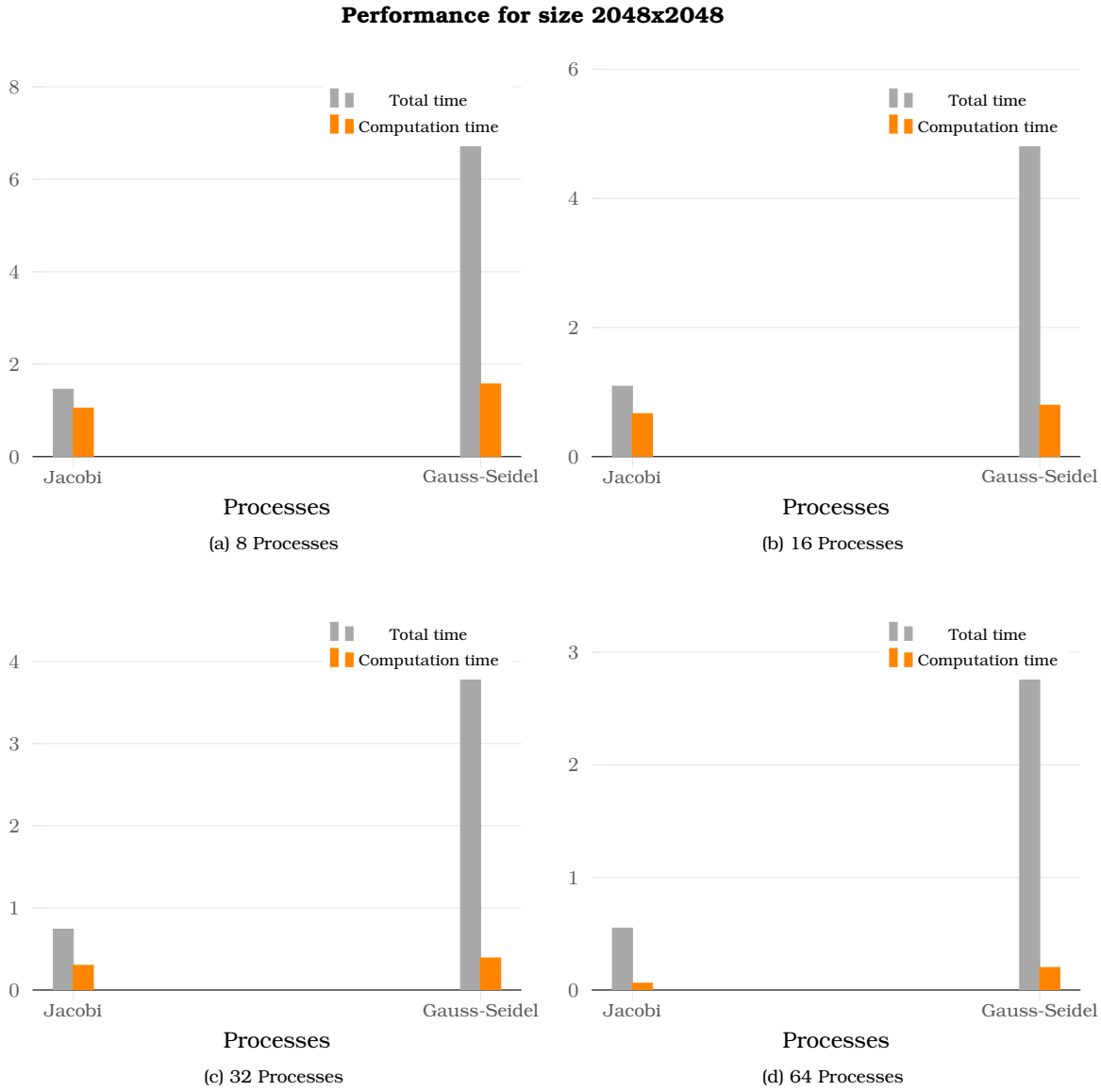


Figure 25: Heat equation: Speedup of each algorithm for problem size 2048x2048



(a) 8 Processes

(b) 16 Processes

(c) 32 Processes

(d) 64 Processes

Για τις περιτπώσεις με μέγεθος προβήματος 4096x4096 και 6144x6144 διακρίνουμε την εξης ίδια συμπεριφορά: όπως και προηγουμένως και οι δυο αλγόριθμοι παρουσιάζουν speedup, με σημαντικότερο αυτό του Jacobi, Ωστόσο μικρότερο απο αυτό στο size 2048x2048. Αυτό οφείλεται, παρατηρώντας τα διαγράμματα χρόνου, στην αύξηση του computational χρόνου, ενδεχομένως λόγω του ότι για μεγαλύτερα μεγέθη πίνακα από έναν αριθμό processes και μετά (από 16 διεργασίες) οι caches σε κάθε κόμβο δεν είναι ικανές να "φυλάξουν" όλο τον local πίνακα, ώστε να συμβαίνουν cache misses και να υπάρχει γενικότερα ανταγωνισμός στην προσπέλαση της μνήμης. Αντιθέτως, η Gauss-Seidel είναι ήδη σε μεγάλο βαθμό πιο "σειριακή" (ο υπολογισμός μιας τιμής απαιτεί να έχουν προηγηθεί δύο γειτονικοί), ώστε να μην παρατηρείται αλλαγή στο speedup.

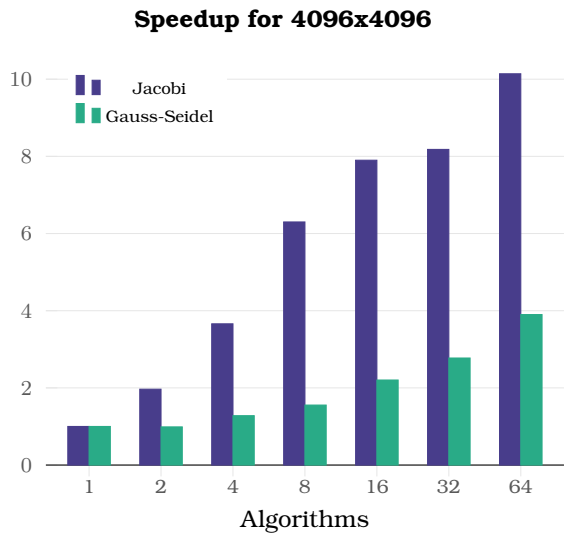
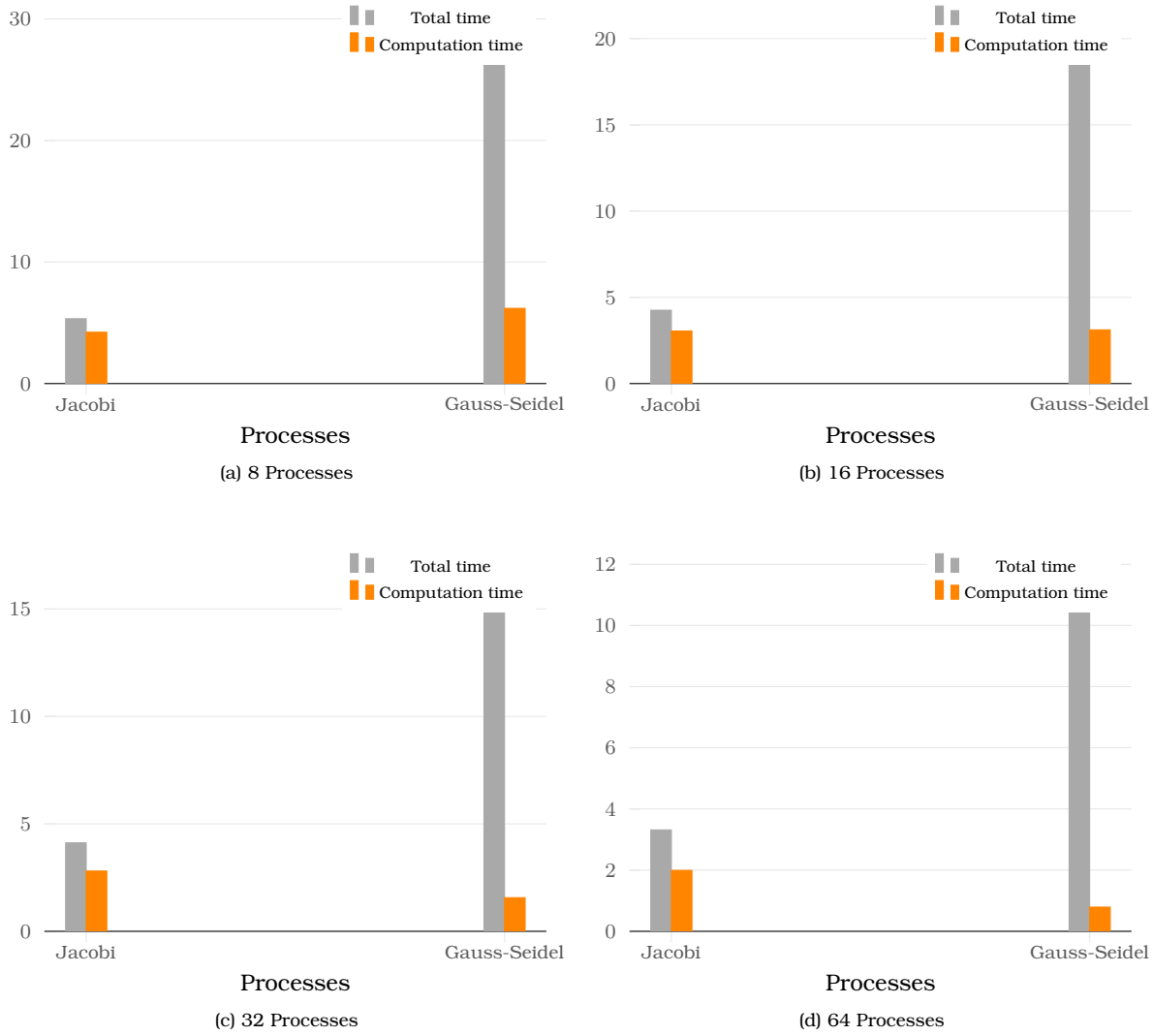


Figure 26: Heat equation: Speedup of each algorithm for problem size 4096x4096



(a) 8 Processes

(b) 16 Processes

(c) 32 Processes

(d) 64 Processes

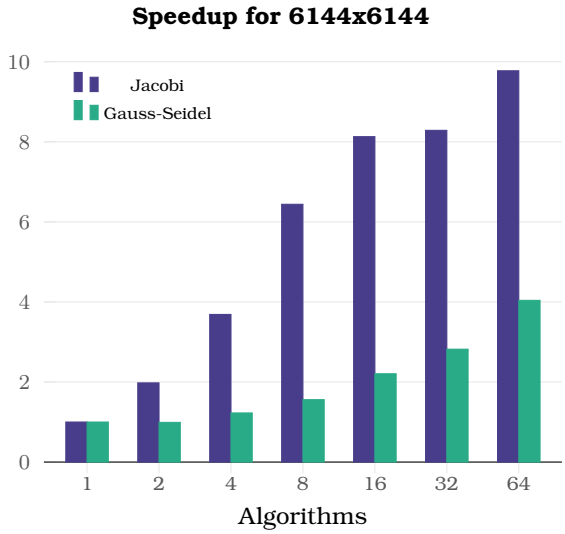
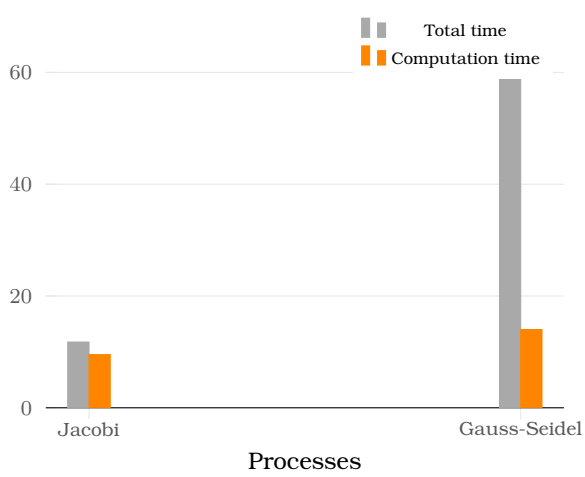
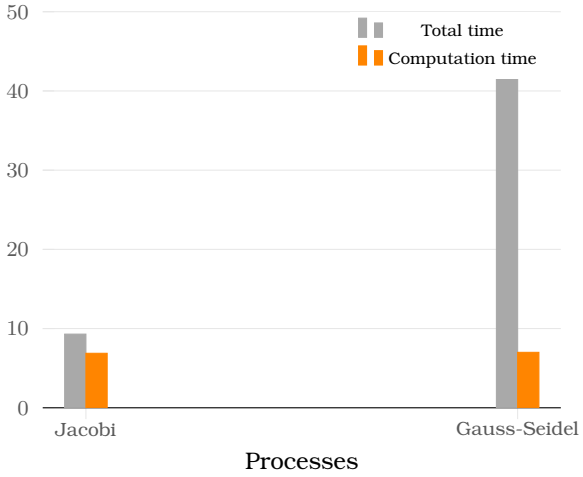


Figure 27: Heat equation: Speedup of each algorithm for problem size 6144x6144

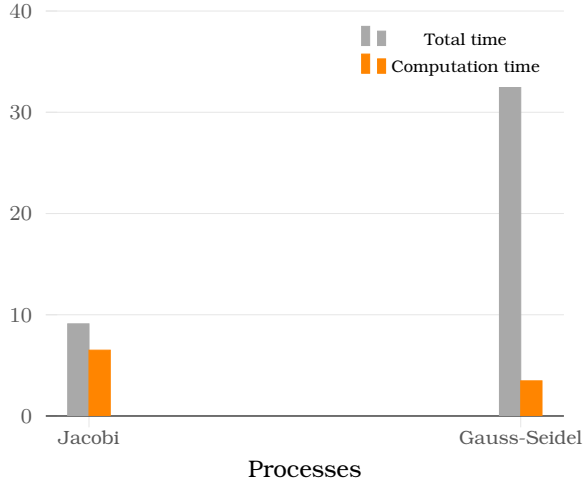
Performance for size 6144x6144



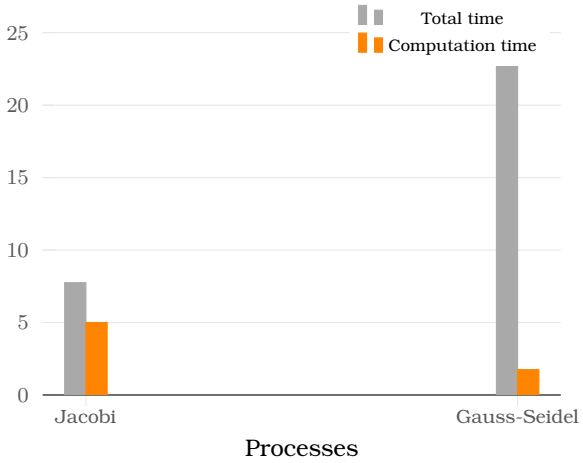
(a) 8 Processes



(b) 16 Processes



(c) 32 Processes



(d) 64 Processes